

SCUBEX: HERRAMIENTA Y RED SOCIAL PARA BUCEADORES

IVÁN FERNÁNDEZ LIMÁRQUEZ

Trabajo fin de Grado

Supervisado por Dr. Pablo Trinidad Martín-Arroyo



Universidad de Sevilla

abril 2026

Publicado en abril 2026 por

Iván Fernández Limárquez

Copyright © MMXXVI

<http://www.lsi.us.es/~trinidad>

ivaferlim@alum.us.es

Pon aquí cuestiones acerca del copyright

Yo, D. Iván Fernández Limárquez con NIF número 77015962W,

DECLARO

mi autoría del trabajo que se presenta en la memoria de este trabajo fin de grado que tiene por título:

Scubex: Herramienta y Red social para Buceadores

Lo cual firmo,

Fdo. D. Iván Fernández Limárquez
en la Universidad de Sevilla
30/04/2026

Para mi familia, amigos y para el mar que ha inspirado este proyecto.



AGRADECIMIENTOS

Agradecer a mi familia, amigos y conocidos que han probado mi aplicación. Con los que pese a haber sido tan pesado, han tenido la paciencia de seguir mis interesantísimas discusiones sobre temas como: Que color es mas fácil de ver, donde es objetivamente mas intuitivo que vaya un botón o, por vigesimotercera vez, escucharme sobre lo chula que esta la animación de inicio.

Especial mención a mi madre, quien sabe cuanto amo el mar y me ha impulsado y apoyado en todo momento.

RESUMEN

Scubex es una red social y herramienta pensada para buceadores que necesiten conocer diversos aspectos relacionados con las zonas de inmersión. La aplicación permite ver las especies en la zona y datos sobre ellas, condiciones climatológicas y compartir experiencias sobre la misma. Mediante publicaciones los usuarios podrán conectar unos con otros y conocer de la mano de demás buceadores si esa zona es interesante o no.

ÍNDICE GENERAL

I	Introducción	1
1.	Contexto	3
1.1.	Redes sociales y exploración marina	4
1.2.	Aplicaciones y tecnologías similares	4
2.	Objetivos	7
2.1.	Motivación	8
2.2.	Listado de objetivos	8
II	Organización del proyecto	11
3.	Metodología	13
3.1.	Estructura organizacional del proyecto	14
3.2.	Metodología de desarrollo	14
3.2.1.	Justificación de Feature-Driven Development (FDD)	14
3.2.2.	Adaptación del ciclo FDD para Scubex	15
3.2.3.	Herramientas de desarrollo	15
3.3.	Seguimiento y control	16
4.	Planificación	17

4.1. Resumen temporal del proyecto	18
4.2. Planificación inicial	18
4.3. Informe de tiempos del proyecto	18
5. Costes	21
5.1. Resumen de costes del proyecto	22
5.2. Costes de personal	22
5.2.1. Desarrollador Full-Stack (Arquitectura e Implementación)	22
5.2.2. Becario Junior (Redacción de Documentación)	23
5.2.3. Total costes de personal	23
5.3. Costes materiales	24
5.3.1. Amortización de equipamiento informático	24
5.3.2. Despliegue en la nube	24
5.3.3. Total costes materiales	25
5.4. Costes indirectos	25
5.5. Coste total del proyecto	25
III Desarrollo del proyecto	27
6. Arranque	29
6.1. Lista de características	30
6.2. Diseño arquitectónico	31
6.2.1. Stack Tecnológico	31
6.2.2. Estrategia de entornos	32
6.2.3. Justificación de MapLibre GL JS + MapTiler	33
6.2.4. Integración de APIs Externas	33

7. Iteración 1: Inicio del MVP de Scubex	35
7.1. Características desarrolladas esta iteración	36
7.2. Diseño	40
7.3. Implementación	40
7.4. Pruebas	49
7.4.1. SpeciesServiceTest	49
7.4.2. JwtServiceTest	50
7.4.3. AuthControllerTest	50
7.5. Despliegue	51
8. Iteración 2: Sistema Climático	53
8.1. Características desarrolladas esta iteración	54
8.2. Diseño	56
8.3. Implementación	56
8.4. Pruebas	62
8.5. Despliegue	62
9. Iteración 3: Publicaciones	63
9.1. Características desarrolladas esta iteración	64
9.2. Diseño	66
9.3. Implementación	66
9.4. Pruebas	71
9.5. Despliegue	72
10. Iteración 4: Interacciones y Red Social	75
10.1. Características desarrolladas esta iteración	76

10.2. Diseño	80
10.3. Implementación	80
10.4. Pruebas	85
10.5. Despliegue	87
 IV Cierre del proyecto	 89
 11. Manual de usuario	 91
11.1. Sección libre	92
 12. Conclusiones	 93
12.1. Informe post-mortem	94
12.1.1. Lo que ha ido bien	94
12.1.2. Lo que ha ido mal	94
12.1.3. Discusión	94
12.2. Trabajos futuros	94
 V Appendices	 95
 A. Software Product Lines	 97
A.1. Software Product Lines	98
A.2. Feature Models	98
A.3. Automated Analysis of Feature Models	100
A.3.1. Scope	100
A.4. Dynamic Software Product Lines (DSPL)	103
A.5. Hypothesis and Objectives	103

Referencias bibliográficas

106

ÍNDICE DE FIGURAS

7.1. Diagrama UML de diseño para la iteración 1	41
8.1. Diagrama UML de diseño para la iteración 2	57
9.1. Diagrama UML de diseño para la iteración 3	66
10.1. Diagrama UML de diseño para la iteración 4	80
A.1. An example of a Home Integration System	99
A.2. A different view on AAFM distinguishing between information extrac- tion and explanatory operations	101

ÍNDICE DE CUADROS

4.1. Tabla resumen de tiempos y planificación	18
4.2. Planificación temporal de iteraciones	18
4.3. Planificación temporal de iteraciones	19
5.1. Tabla resumen de costes del proyecto	22
5.2. Coste total del proyecto Scubex	25
6.1. Estrategia de entornos del proyecto	32
6.2. Resumen de APIs externas	34
7.1. Análisis de valor aportado: Mapa Interactivo	37
7.2. Análisis de valor aportado: Escáner	38
7.3. Análisis de valor aportado: Autenticación	39
7.4. Memorando técnico 001: Geometría WKT	42
7.5. Memorando técnico 002: Filtrado de Calidad Visual	43
7.6. Memorando técnico 003: Caché L1 — CachedScan	44
7.7. Memorando técnico 004: Caché L2 — SpeciesEnrichmentCache	45
7.8. Memorando técnico 005: Autenticación OAuth2 + JWT	48
8.1. Análisis de valor aportado: Panel de Clima	55
8.2. Memorando técnico 006: WeatherResponse	58
8.3. Memorando técnico 007: Evaluación de Condiciones	59

8.4. Memorando técnico 008: Caché de Clima	60
9.1. Análisis de valor aportado: Publicaciones Geolocalizadas	65
9.2. Memorando técnico 009: Filtrado por Viewport del Mapa	67
9.3. Memorando técnico 010: Geocoding Inverso con Nominatim	68
9.4. Memorando técnico 011: Almacenamiento de Imágenes como BLOB . .	70
10.1. Análisis de valor aportado: Interacciones con Publicaciones	77
10.2. Análisis de valor aportado: Seguimiento de Usuarios y Perfiles Públicos	78
10.3. Análisis de valor aportado: Centro de Notificaciones	79
10.4. Memorando técnico 012: Likes, Guardados y Comentarios	81
10.5. Memorando técnico 013: Notificaciones y Menciones	83
10.6. Memorando técnico 014: Centro de Notificaciones	84
A.1. Most frequently used explanatory operations and their corresponding information extraction operations	105

LISTA DE TAREAS PENDIENTES

■ To Abductive Section in 2.1	98
Figura: A feature model example	99
■ To Abductive Intro	100

PARTE I

INTRODUCCIÓN

CONTEXTO

The sea, once it casts its spell, holds one in its net of wonder forever.

*Jacques Yves Cousteau,
Oceanographer*

Vamos a dar una visión general del contexto en el que se desarrolla el proyecto, incluyendo el estado actual de la industria y el porqué del mismo.

1.1 REDES SOCIALES Y EXPLORACIÓN MARINA

Desde pequeño he vivido rodeado de mar, pero es solo ahora cuando he descubierto lo mucho que me gusta el buceo. Sin embargo, este viene con una serie de retos y problemas: ¿Dónde me sumerjo?, ¿Hará buen clima debajo del agua?, ¿Podré llegar a la zona con mi equipo o hará mucho viento? y, si voy, ¿Qué especies podré ver?

Con este proyecto busco dar una solución a estos problemas desde dos perspectivas. Estadística y personal, el usuario podrá basarse en cientos de datos ya recopilados de una zona, tomado de una base de datos que lleva creada y actualizándose desde el 1970. O simplemente podrá basarse en la experiencia de otros buceadores en esa misma zona.

1.2 APLICACIONES Y TECNOLOGÍAS SIMILARES

No tendría sentido desarrollar un proyecto que ya existe, por ello he buscado aplicaciones que hagan algo similar a la propuesta de Scubex. Que básicamente se basa en dar respuesta a las preguntas preguntas que he mencionado, para ello hay que dar información de: Que condiciones climatológicas hay en una zona, que especies marinas podrás ver y como está el mar ese día concreto en esa zona específica. Esto debe hacerse usando tanto datos recopilados a lo largo de los años de una zona, como dejando a los usuarios compartir sus experiencias personales. Buscando aplicaciones que den respuesta a estas preguntas he encontrado tres:

La primera es **Dive Mate**: Una aplicación que te muestra especies que has visto tú en una zona y que te da estimaciones climatológicas precisas. Sin embargo, la interfaz es tosca. Y no es posible compartir experiencias personales, un usuario no puede interactuar con otros ni registrar sus propias observaciones.

La segunda es **iNaturalist**: Una aplicación que te permite ver, en un mapa, avistamientos de especies en esa zona. Sus fotos e información de avistamientos son increíblemente ricas y útiles. Sin embargo, no está centralizada en buceadores y no incluye información climatológica para el mar de la zona.

La última es **DiveApp**: Una aplicación orientada a los avistamientos y preparación de viajes que incluye componentes de red social. Tiene el problema contrario a Dive Mate, incluye experiencia personal a modo de publicaciones y clima sobre la zona. Pero sin embargo no incluye datos ya recopilados sobre la zona en la que pretendes

sumergirte.

Como conclusión, hay aplicaciones que ofrecen funcionalidades parecidas a las que esta aplicación proporciona, sin embargo, ninguna las centraliza. Ni están implementadas de una forma intuitiva, ni con tecnologías modernas. En este hueco de mercado encaja Scubex. Por ello me he decidido a hacer este proyecto

OBJETIVOS

If you don't know where you are going, you'll end up someplace else.

*Yogi Berra,
Baseball player*

Visto el contexto, formalizaremos los objetivos de la app de forma concreta. ¿Hacia quien esta dirigida la aplicación? ¿Qué funcionalidades tendrá? ¿Qué tecnologías se van a usar? ¿Qué se va a conseguir con el proyecto?

2.1 MOTIVACIÓN

Como ya hemos comentado, la idea de Scubex es centralizar funcionalidades para el nicho del buceo. Para lograrlo se han planteado esta serie de objetivos que, una vez cumplidos, darán como resultado la aplicación deseada.

2.2 LISTADO DE OBJETIVOS

Objetivo 1. Implementar un sistema de autenticación de usuarios

Desarrollar un sistema de autenticación seguro, permitiendo a los usuarios registrarse e iniciar sesión de forma sencilla. El sistema debe guardar los perfiles de usuario y datos tales como: nombre, foto y publicaciones.

Objetivo 2. Crear un mapa interactivo de exploración marina

Integrar un mapa interactivo con controles de zoom y navegación que permita a los usuarios explorar zonas de buceo de forma intuitiva y fluida.

Objetivo 3. Integrar datos científicos de biodiversidad marina

Desarrollar un escáner de especies que consulte APIs científicas para identificar fauna marina en una zona determinada.

Objetivo 4. Proporcionar información climatológica y oceanográfica en tiempo real

Ofrecer datos actualizados sobre condiciones climatológicas de la zona: viento, temperatura del agua, corrientes marinas, oleaje y demás datos atmosféricos relevantes para la planificación de inmersiones en una zona de inmersión.

Objetivo 5. Implementar un sistema de registro de publicaciones

Permitir a los usuarios publicar sus inmersiones en el mapa interactivo con título, descripción opcional, fotografía y coordenadas geográficas. El autor puede editar y eliminar sus propias publicaciones, y cada publicación es compartible mediante enlace directo.

Objetivo 6. Crear un sistema de interacción social sobre publicaciones

Desarrollar un sistema de interacciones sobre publicaciones: likes, guardado para acceso posterior y comentarios con menciones a otros usuarios. Las menciones generan notificaciones al usuario mencionado.

Objetivo 7. Implementar una red social de seguimiento entre usuarios

Desarrollar perfiles públicos navegables con número de seguidores y seguidos,

posibilidad de seguir o dejar de seguir a cualquier usuario, y acceso al historial de publicaciones de cada perfil. Los perfiles son compartibles mediante enlace directo.

Objetivo 8. Implementar un sistema de notificaciones

Proporcionar notificaciones al usuario sobre eventos relevantes: nuevos seguidores, likes, comentarios en sus publicaciones y menciones. Las notificaciones son gestionables, pudiendo borrarlas o marcarlas como leídas.

PARTE II

ORGANIZACIÓN DEL PROYECTO

METODOLOGÍA

Don't reinvent the wheel, just realign it.

*Anthony J. D'Angelo,
Escritor motivacional*

***E**ste capítulo describe el marco metodológico adoptado para el desarrollo de Scubex, basado en Feature-Driven Development (FDD) con adaptaciones específicas para un proyecto académico individual.*

3.1 ESTRUCTURA ORGANIZACIONAL DEL PROYECTO

Scubex se ha desarrollado como Trabajo Fin de Grado de forma individual, asumiendo las siguientes responsabilidades:

- **Arquitecto de Software:** Diseño de la arquitectura cliente-servidor y selección del stack tecnológico.
- **Desarrollador Full-Stack:** Implementación del backend (Spring Boot) y frontend (React + Vite).
- **Documentador Técnico:** Redacción de la memoria académica y especificación de APIs mediante OpenAPI (Swagger).

El tutor del proyecto ha proporcionado la supervisión técnica y académica, validando decisiones arquitectónicas y avances iterativos.

3.2 METODOLOGÍA DE DESARROLLO

3.2.1 Justificación de Feature-Driven Development (FDD)

Se ha adoptado **Feature-Driven Development** como metodología central debido a sus características idóneas para proyectos de alcance medio con requisitos funcionales bien definidos:

- **Orientación a características:** Cada funcionalidad (escáner de especies, integración climática, red social) se desarrolla como una única unidad. Lo que significa que se realizará el código y hasta que no esté documentada y acabada, no se pasará a la siguiente.
- **Trazabilidad:** Al forzar la compleción completa de una característica hay una relación estrecha y directa entre requisitos, código y documentación. Esto hace que sea fácil trazar un camino claro del avance del proyecto.
- **Escalabilidad:** Gracias a este enfoque es relativamente añadir nuevas características en el futuro. Ya que simplemente hay que añadir una característica a la lista y tratarla como esa unidad que previamente hemos mencionado.

En resumen, esta metodología permite estructurar el desarrollo con una lista de características claras y priorizadas. Lo que ayuda a mantener una organización coherente y simple en un equipo pequeño o individual.

3.2.2 Adaptación del ciclo FDD para Scubex

El modelo clásico de FDD consta de 5 fases. Para este proyecto académico se ha adaptado la secuencia, unificando la fase de diseño y añadiendo una fase de documentación:

1. **Desarrollar la lista de características:** Identificación y priorización de funcionalidades nucleares (Gestión de usuarios OAuth2, Escáner de especies, Exploración climática, Red social).
2. **Construir un modelo global:** Diseño de la arquitectura del sistema a partir de las características identificadas (capítulo de Arranque).
3. **Planificar por característica:** Estimación tecnológica, limitaciones y consecuente diseño del flujo de datos.
4. **Construir por característica:** Implementación incremental con validación mediante pruebas constantes.
5. **Documentar la característica:** Redacción académica de cada característica en LaTeX, incluyendo justificación de decisiones técnicas y lecciones aprendidas.

3.2.3 Herramientas de desarrollo

- **Control de versiones:** Git + GitHub (commits atómicos por característica).
- **Gestión de dependencias:** Maven (backend), npm (frontend).
- **Testing:** JUnit 5 + Mockito (backend), Vitest (frontend).
- **Documentación de API:** Swagger/OpenAPI 3.0 (generación automática de contratos REST).
- **Compilación:** Vite (desarrollo frontend con HMR), Spring Boot Maven Plugin (empaquetado JAR).

- **Despliegue:** Vercel (frontend, despliegue automático por rama con previews) + Railway (backend Spring Boot y PostgreSQL, entornos de preproducción y producción independientes).

3.3 SEGUIMIENTO Y CONTROL

El progreso del proyecto se ha monitorizado mediante:

- **Archivo TODO:** Lista priorizada de tareas pendientes con urgencias marcadas (!).
- **Commits semánticos:** Mensajes (casi siempre) estructurados (feat, fix, docs, refactor) para trazabilidad histórica.
- **Reuniones de seguimiento:** Sesiones cada tres semanas con el tutor para validar decisiones arquitectónicas y resolver bloqueos técnicos.
- **Documentación incremental:** Redacción de capítulos inmediatamente tras completar cada característica dentro de la aplicación.

Métricas de avance:

- Características completadas.
- Cobertura de tests unitarios (objetivo: >70% en lógica de negocio).

PLANIFICACIÓN

Divide y vencerás · El arte de la guerra

*Sun Tzu (544–496 a.C.),
Maestro de guerra y escritor*

*E*n este capítulo se exponen las divisiones en sprint planificadas para la realización del proyecto.

4.1 RESUMEN TEMPORAL DEL PROYECTO

Resumen del proyecto	
Fecha de inicio	13/10/2025
Fecha de fin	XX/XX/2026
Periodicidad de las revisiones	3 semanas
Carga de trabajo semanal	12 horas
Horas totales previstas	285 horas
Horas finales	293 horas

Cuadro 4.1: Tabla resumen de tiempos y planificación

4.2 PLANIFICACIÓN INICIAL

Aquí un desglose de las iteraciones, comienzo y fin de cada una:

Resumen de iteraciones	
Iteración 1	13/10/25 a 13/03/26
Iteración 2	13/03/26 a 03/04/26
Iteración 3	04/04/26 a 24/04/26
Iteración 4	25/04/26 a 08/05/26
...	dd/mm/aa a dd/mm/aa

Cuadro 4.2: Planificación temporal de iteraciones

Cabe destacar que la primera iteración es más larga que las siguientes. Esto se debe a que el proyecto se inició en octubre, mientras que la documentación comenzó en febrero. Dado que hubo un parón entre ambas fechas, se decidió que la primera iteración abarcara todo ese tiempo, para no perder el trabajo realizado durante ese periodo. Las siguientes iteraciones se planificaron con una duración de 3 semanas cada una, para mantener un ritmo constante de trabajo y permitir revisiones periódicas del progreso del proyecto.

4.3 INFORME DE TIEMPOS DEL PROYECTO

Lo mismo que el anterior pero con datos reales. Ver Tabla §4.3.

Resumen de iteraciones	
Iteración 1	13/10/25 a 13/03/26
Iteración 2	13/03/26 a 03/04/26
Iteración 3	04/04/26 a 24/04/26
Iteración 4	25/04/26 a 08/05/26
...	dd/mm/aa a dd/mm/aa

Cuadro 4.3: Planificación temporal de iteraciones

Tras la primera iteración, no hubo más descoordinación entre las fechas planificadas y las reales. Se cumplió con el calendario establecido.

COSTES

Here Comes the Money

*Jim Johnston y Naughty by Nature,
Escritores y Raperos*

***E**n este capítulo se realiza el desglose de los costes asociados al proyecto, basándose en los salarios reales de puestos tecnológicos.*

5.1 RESUMEN DE COSTES DEL PROYECTO

Resumen del proyecto	
Costes de personal	14.601,17 €
Desarrollador Full-Stack	9.522,50 €
Becario Documentación	5.078,67 €
Costes materiales	67,23 €
Costes indirectos	1.466,84 €
TOTAL	16.135,24 €

Cuadro 5.1: Tabla resumen de costes del proyecto

5.2 COSTES DE PERSONAL

Para el cálculo de los costes de personal se han consultado los datos de la **Guía Salarial 2026** de Manfred¹, que recopila información salarial de más de 120.000 profesionales de tecnología en España.

Dado que el desarrollo de Scubex ha requerido asumir múltiples roles y tareas, se han considerado dos perfiles:

5.2.1 Desarrollador Full-Stack (Arquitectura e Implementación)

Se ha considerado como referencia el salario de **Desarrollador Full-Stack** con experiencia de 5-10 años, cuyo rango salarial se sitúa entre 41.000-50.000€ anuales según dicha guía. Se ha tomado un valor medio de **45.000€ brutos anuales**.

Cálculo proporcional al tiempo dedicado

La carga de trabajo estimada para el TFG es de **293 horas**, frente a una jornada laboral anual estándar de **1.800 horas**. Por tanto, el coste proporcional se calcula como:

$$\text{Salario proporcional} = 45,000\text{€} \times \frac{293}{1,800} = 7,325\text{€} \quad (5.1)$$

¹<https://www.getmanfred.com/blog/guia-salarial-2026-salarios-en-tecnologia-espana-manfred>

Costes sociales

A este salario bruto se le añaden los **costes sociales** (Seguridad Social, contingencias comunes, desempleo, formación profesional, etc.) que suponen aproximadamente el **30 %** del salario bruto:

$$\text{Costes sociales} = 7,325\text{€} \times 0,30 = 2,197,50\text{€} \quad (5.2)$$

Total Desarrollador Full-Stack

$$\text{Total} = 7,325\text{€} + 2,197,50\text{€} = 9,522,50\text{€} \quad (5.3)$$

5.2.2 Becario Junior (Redacción de Documentación)

Para la redacción de la memoria académica y documentación técnica se ha considerado el perfil de un **becario junior** con experiencia menor a 2 años. Según la guía salarial, los desarrolladores junior se sitúan en el rango de 20.000-30.000€ anuales. Se ha tomado un valor de **24.000€ brutos anuales**.

Cálculo proporcional al tiempo dedicado

De la misma forma que se ha calculado anteriormente:

$$\text{Salario proporcional} = 24,000\text{€} \times \frac{293}{1,800} = 3,906,67\text{€} \quad (5.4)$$

Costes sociales

$$\text{Costes sociales} = 3,906,67\text{€} \times 0,30 = 1,172\text{€} \quad (5.5)$$

Total Becario

$$\text{Total} = 3,906,67\text{€} + 1,172\text{€} = 5,078,67\text{€} \quad (5.6)$$

5.2.3 Total costes de personal

$$\text{Total} = 9,522,50\text{€} + 5,078,67\text{€} = 14,601,17\text{€} \quad (5.7)$$

5.3 COSTES MATERIALES

5.3.1 Amortización de equipamiento informático

Según el criterio de amortización fiscal establecido por la Agencia Tributaria española, los equipos informáticos se amortizan a un **máximo del 25 % anual** (vida útil de 4 años).

Para el desarrollo del proyecto se ha utilizado un portátil con un coste de adquisición de **1.200€**:

- **Amortización anual:** $1,200€ \times 0,25 = 300€$
- **Amortización proporcional** (293h / 1.800h): $300€ \times \frac{293}{1,800} = 48,83€$

5.3.2 Despliegue en la nube

El proyecto se encuentra desplegado en producción y preproducción mediante dos servicios:

- **Railway** (backend Spring Boot + PostgreSQL): Plan Hobby a 5 USD/mes (~4,60 €/mes). Este plan incluye 5 USD de créditos de uso mensuales; superado ese umbral, se factura por consumo real de recursos. Actualmente debido al tráfico bajo-moderado, el crédito mensual de 5 USD resulta suficiente.
- **Vercel** (frontend React/Vite): Plan Hobby gratuito, sin coste.

Coste estimado de despliegue durante el desarrollo del TFG (4 meses):

$$\text{Railway} = 4,60 \text{ €/mes} \times 4 \text{ meses} = 18,40 \text{ €} \quad (5.8)$$

$$\text{Total despliegue} = 18,40 \text{ €} + 0 \text{ € (Vercel)} = 18,40 \text{ €} \quad (5.9)$$

En un escenario de producción real con mayor tráfico, los costes de Railway podrían aumentar proporcionalmente al uso de CPU, RAM y almacenamiento consumidos más allá del crédito incluido. En este caso podría crearse un plan personalizado, que incluya las necesidades concretas de la aplicación.

5.3.3 Total costes materiales

$$\text{Total} = 48,83 \text{ €} + 18,40 \text{ €} = 67,23 \text{ €} \quad (5.10)$$

5.4 COSTES INDIRECTOS

Los costes indirectos incluyen gastos generales asociados al desarrollo del proyecto que no se imputan directamente: electricidad, conexión a internet, espacio de trabajo, cuotas mensuales de asistentes de IA, etc.

Se ha tomado un valor medio del 10 % de la suma de costes de personal y materiales:

$$\text{Costes indirectos} = (14,601,17 \text{ €} + 67,23 \text{ €}) \times 0,10 = 1,466,84 \text{ €} \quad (5.11)$$

5.5 COSTE TOTAL DEL PROYECTO

Desglose de costes	
Concepto	Importe
Costes de personal	14.601,17€
Desarrollador Full-Stack	9.522,50€
Becario Documentación	5.078,67€
Costes materiales	67,23€
Amortización equipamiento	48,83€
Despliegue Railway (4 meses)	18,40€
Costes indirectos	1.466,84€
TOTAL PROYECTO	16.135,24€

Cuadro 5.2: Coste total del proyecto Scubex

PARTE III

DESARROLLO DEL PROYECTO

ARRANQUE

The best way to get started is to quit talking and begin doing.

Walt Disney (1901–1966),

Empresario y cineasta

***E**ste capítulo define la lista inicial de características priorizadas del proyecto Scubex y, a partir de ellas, la arquitectura base necesaria para soportarlas. Se sigue el orden natural del ciclo FDD: primero se establece qué se va a construir, y solo entonces se diseña cómo.*

6.1 LISTA DE CARACTERÍSTICAS

Siguiendo la fase 1 de FDD (Desarrollar lista de características) y los objetivos definidos para el proyecto, se han identificado las siguientes funcionalidades nucleares priorizadas por valor para el usuario:

1. **Gestión de Usuarios (OAuth2):** Autenticación delegada mediante Google y persistencia de perfil (nombre, foto). El usuario puede personalizar su nombre y foto de perfil desde la página de perfil, con soporte para eliminar la cuenta.
2. **Mapa Interactivo:** Visualización de un mapa navegable mediante MapLibre GL JS como superficie central de interacción, con búsqueda de lugares integrada (Nominatim).
3. **Escáner de Especies:** Identificación de fauna marina en una zona mediante radio de búsqueda y validación cruzada de APIs científicas (OBIS + iNaturalist), con sistema de caché adaptativo (TTL 48h) para reducir latencia y llamadas externas.
4. **Datos Climáticos y Oceanográficos:** Condiciones de la zona bajo demanda mediante Open-Meteo (viento, temperatura del agua, corrientes, olas, datos atmosféricos), con evaluación de aptitud para el buceo y caché de 30 minutos en el backend.
5. **Registro de Publicaciones:** Publicación de inmersiones en el mapa con título, descripción, fotografía y geolocalización inversa (nombre del lugar mediante Nominatim). Edición y borrado por el propio autor. Cada publicación genera un enlace compartible (`/map?pub={id}`).
6. **Interacciones Sociales:** Sistema de likes, guardados y comentarios sobre publicaciones. Soporte de menciones en comentarios mediante la sintaxis `@[Nombre] (email)` con notificaciones automáticas al mencionado. El enlace compartible de cada publicación es accesible mediante un botón que usa la Web Share API en móvil y copia al portapapeles en escritorio.
7. **Red Social y Seguimiento de Usuarios:** Perfiles públicos por usuario con recuento de seguidores y seguidos, posibilidad de seguir/dejar de seguir, y vista de publicaciones de cada perfil. Página de perfil propio con mis publicaciones y publicaciones guardadas. Cada perfil dispone de enlace compartible (`/user/{email}`) accesible desde un botón de compartir en la página de perfil.

8. **Sistema de Notificaciones:** Notificaciones en tiempo real (campana en el header) para eventos de seguimiento, like, comentario y mención. Las notificaciones se marcan como leídas al abrirlas o se eliminan permanentemente con swipe (móvil) o botón (escritorio).

Justificación de priorización: La lista anterior refleja el orden de implementación planificado, de mayor a menor prioridad. A continuación se justifica ese orden:

- La autenticación se implementa primero ya que el resto de funcionalidades (avistamientos, perfil, red social) dependen de la identidad del usuario.
- El mapa es la superficie central de la aplicación: sin él no tiene sentido geolocalizar especies, condiciones climáticas ni avistamientos.
- El escáner de especies, los datos climáticos y el registro de publicaciones constituyen el MVP funcional mínimo. Como se planteó en el capítulo de Contexto, el objetivo es que el usuario pueda tomar decisiones apoyándose tanto en datos objetivos como en experiencias reales: el escáner y el clima aportan la primera dimensión; las publicaciones, la segunda, al permitir consultar las inmersiones documentadas por otros buceadores en la misma zona.
- Las funcionalidades de comunidad (likes, comentarios, seguimiento entre usuarios, notificaciones) se implementan sobre ese MVP: convierten la herramienta en una red social de buceadores, pero no son necesarias para que la propuesta de valor principal sea utilizable.

6.2 DISEÑO ARQUITECTÓNICO

Con las características identificadas, el sistema requiere una arquitectura capaz de orquestar múltiples APIs externas, gestionar identidad de usuario y servir una interfaz cartográfica interactiva. Se ha optado por una **arquitectura cliente-servidor desacoplada** con patrón Gateway de APIs.

6.2.1 Stack Tecnológico

- **Backend:** Spring Boot 4 (Java 21). Actúa como Gateway de APIs científicas, orquestador de peticiones asíncronas y gestor de autenticación.

- **Frontend:** React 18 + Vite. Con Hot Module Replacement (HMR) para desarrollo ágil. Gestión de estado reactiva mediante MobX `makeAutoObservable`, con stores separadas por dominio (especies, clima, publicaciones, usuario, mapa).
- **Cartografía:** MapLibre GL JS. Motor de renderizado vectorial para animaciones fluidas y bajo consumo de datos.
- **Base de Datos:** H2 en memoria para el entorno de desarrollo y tests unitarios. PostgreSQL 16 para preproducción y producción, gestionado como servicio en Railway.
- **Testing:** JUnit 5 + Mockito para pruebas unitarias del backend. H2 en memoria como base de datos de test.
- **Seguridad:** Spring Security con OAuth2 Client para Google Login.
- **Documentación de API:** OpenAPI 3.0 (Swagger UI) con anotaciones automáticas.
- **Despliegue:** Railway (backend + PostgreSQL) y Vercel (frontend).

6.2.2 Estrategia de entornos

Se han definido tres entornos que reflejan el ciclo de vida del software desde el desarrollo local hasta la puesta en producción:

Estrategia de entornos de Scubex			
Entorno	Infraestructura	Base de datos	Rama Git
Desarrollo	Local	H2 en memoria	Cualquiera
Preproducción	Railway + Vercel	PostgreSQL (Railway)	develop
Producción	Railway + Vercel	PostgreSQL (Railway)	main

Cuadro 6.1: Estrategia de entornos del proyecto

Justificación de la separación entre plataformas:

- **Vercel** gestiona el frontend React/Vite: Se despliega automáticamente en cada push. Se cuenta con previews por rama sin configuración adicional, más que establecer las direcciones de backend de producción y preproducción.
- **Railway** gestiona el backend Spring Boot y PostgreSQL en el mismo proyecto. Permite tener varios entornos distintos de una misma aplicación desplegados simultáneamente, lo que la hace idónea para combinarla con Vercel.
- La separación permite desplegar frontend y backend de forma independiente. Gracias a esto pueden escalarse independientemente y no solo eso. Debido a la separación se cuenta con más memoria de los planes gratuitos de ambas aplicaciones, reduciendo costes.

La configuración CORS se ha establecido manualmente mediante la variable `CORS_ALLOWED_ORIGINS` en Railway, y `VITE_API_BASE_URL` en Vercel, configuradas con las respectivas URL del backend correspondientes a preproducción y producción.

6.2.3 Justificación de MapLibre GL JS + MapTiler

La solución cartográfica se compone de dos piezas complementarias:

- **MapLibre GL JS** actúa como motor de renderizado: biblioteca open source (fork de Mapbox GL JS) que renderiza estilos vectoriales mediante WebGL. Permite animaciones fluidas, rotación e inclinación del mapa, y menor consumo de datos frente a tiles rasterizados. Al ser open source, no impone restricciones de licencia sobre el código.
- **MapTiler** provee los tiles y el estilo visual del mapa a través de su API (`VITE_MAPTILER_KEY`). Se ha creado un estilo personalizado (tema marino) alojado en MapTiler Cloud. El plan gratuito cubre de sobra el volumen de peticiones necesario para el proyecto.

La separación entre motor de renderizado (MapLibre) y proveedor de tiles (MapTiler) permite sustituir el proveedor de tiles sin cambiar el código de renderizado, y viceversa.

6.2.4 Integración de APIs Externas

Estrategia de consumo:

APIs externas integradas en Scubex		
API	Proveedor	Datos proporcionados
OBIS	Ocean Biodiversity Information System	Ocurrencias biológicas: nombre científico, coordenadas, taxonomía (phylum), fecha de avistamiento.
iNaturalist	iNaturalist.org	Metadatos multimedia: fotos de especies, nombres comunes (preferred_common_name).
Open-Meteo	Open-Meteo.com	Datos climáticos y oceanográficos: viento, temperatura del agua/aire, altura de olas, corrientes marinas, precipitaciones. API gratuita sin necesidad de clave y con plan gratuito generoso.
Nominatim	OpenStreetMap Foundation	Geocodificación directa e inversa. Directa (/search): buscador de lugares del mapa. Inversa (/reverse): obtención del nombre humano de unas coordenadas (playa, municipio, estado) al crear publicaciones y al visualizar su detalle.

Cuadro 6.2: Resumen de APIs externas

- **OBIS + iNaturalist:** Consultas de especies marinas mediante el botón de escaneo.
- **Open-Meteo:** Consulta bajo demanda al interactuar con el mapa.

Decisiones pendientes que quedaron en esta fase: Quedó pendiente el determinar la estrategia de caching para OBIS que se resolvió en la Iteración 1; Y también el caching de condiciones climáticas que se resolvió en la Iteración 2.

ITERACIÓN 1: INICIO DEL MVP DE SCUBEX

*E*sta iteración entrega parte del MVP funcional de Scubex: el mapa interactivo con escáner de biodiversidad marina y autenticación con Google.

7.1 CARACTERÍSTICAS DESARROLLADAS ESTA ITERACIÓN

1. Mapa interactivo con capas de teselas, batimetría y búsqueda de lugares. Ver Tabla §7.1.
2. Escáner de biodiversidad marina con enriquecimiento OBIS + iNaturalist. Ver Tabla §7.2.
3. Autenticación con Google OAuth2 y emisión de JWT. Ver Tabla §7.3.

Análisis de valor aportado: Mapa Interactivo	
Propuesta	Mapa interactivo con estilo personalizado y representación visual de batimetría. Incluye buscador de lugares y visualización en tiempo real de la zona de escaneo sobre el mapa.
Valor	Proporciona al usuario la interfaz principal de Scubex: localizar zonas de buceo sobre un mapa real con información de batimetría, fijar el centro con un click y visualizar dinámicamente el área de interés antes de lanzar el escáner.
Coste	Integración de la librería de mapas y uso de la API de MapTiler para estilos y batimetría propios. Conexión con Nominatim para la búsqueda de lugares. Requiere clave de API de MapTiler (plan gratuito: 5.000 sesiones/mes).
Opciones	Lista de spots predefinidos: el usuario elige de un catálogo fijo de zonas de buceo conocidas; más simple pero elimina la libertad de explorar cualquier punto. Buscador de texto: el usuario escribe el nombre del lugar y el sistema devuelve las especies de esa zona; se ha descartado por la falta de feedback visual. El mapa aporta orientación geográfica y espontaneidad que estas alternativas no ofrecen.
Riesgos	Límite mensual de sesiones del plan gratuito de MapTiler. Si el estilo base cambia o se elimina en MapTiler Cloud, la aplicación perdería el estilo del mapa. Dependencia de Nominatim (servicio público sin SLA).
Deuda técnica	La capa de batimetría proviene del estilo base de MapTiler; si cambia maptiler, la capa programática dejaría de funcionar y no se verían las diferentes profundidades.

Cuadro 7.1: Análisis de valor aportado: Mapa Interactivo

Análisis de valor aportado: Escáner de Biodiversidad	
Propuesta	Escáner de biodiversidad marina que consulta registros científicos de OBIS y enriquece cada especie con foto y nombre común de iNaturalist.
Valor	Permite descubrir la fauna marina real de una zona validada científicamente (>150M registros OBIS) con representación visual. Al desplegar la tarjeta de cada especie, el usuario puede consultar su rango de profundidad, temperatura, primer y último avistamiento, registros globales y categoría IUCN. La caché reduce la latencia de consultas repetidas sobre la misma zona.
Coste	Integración con dos APIs externas (OBIS e iNaturalist) y diseño de una caché de dos niveles para evitar latencias excesivas. Esfuerzo alto en la primera iteración por la coordinación de múltiples llamadas en paralelo.
Opciones	Buscador de especie: el usuario busca una especie concreta y la app le indica en qué zonas se encuentra; útil si ya sabes lo que buscas. Sin embargo el escáner es más intuitivo y expone todo el rango de especies de una zona sin que el usuario necesite saber de antemano qué buscar.
Riesgos	Rate limit de iNaturalist (100 req/min). Inconsistencias entre bases de datos (especie en OBIS sin entrada en iNaturalist). Latencia acumulada en primer escaneo de una zona sin caché.
Deuda técnica	Paginación pendiente (limitado a las primeras 1000 ocurrencias de OBIS). Ausencia de invalidación manual de caché si los datos cambian antes del TTL.

Cuadro 7.2: Análisis de valor aportado: Escáner

Análisis de valor aportado: Autenticación	
Propuesta	Inicio de sesión con cuenta de Google en un solo clic. El backend verifica la identidad con Google y emite un token propio que autentica las peticiones posteriores del usuario.
Valor	Elimina la fricción de registro manual. El usuario usa su identidad Google ya verificada. El JWT permite autenticar peticiones futuras sin nueva validación contra Google.
Coste	Configuración de credenciales en Google Cloud Console y gestión segura de los secretos de la aplicación. Implementación del filtro de seguridad que valida el token en cada petición.
Opciones	Auth0 / Firebase Auth: evita implementar la lógica JWT, pero añade dependencia de servicio externo y coste. Registro propio con email/contraseña: mayor fricción y necesidad de gestionar contraseñas.
Riesgos	Compromiso de JWT_SECRET: todos los tokens activos quedan expuestos. Expiración de 7 días sin mecanismo de refresco: el usuario debe volver a autenticarse al caducar el token.
Deuda técnica	Sin <i>refresh tokens</i> : al expirar el JWT el usuario debe volver a hacer login. Sin revocación de tokens comprometidos.

Cuadro 7.3: Análisis de valor aportado: Autenticación

7.2 DISEÑO

El diagrama UML de la Fig. §7.1 muestra las relaciones entre los componentes principales del sistema: controladores, servicios, repositorios y entidades de caché. La leyenda del propio diagrama recoge la notación de visibilidad y el código de colores por capa arquitectónica.

Cabe señalar que el método `findBestMatch()` empleado en los pseudocódigos es un alias semántico: el nombre real, generado automáticamente por Spring Data JPA a partir de los criterios de consulta, es `findFirstByRoundedLatAndRoundedLngAndRadiusGreaterThanOrEqualToCreatedAtAfterOrderByRadiusDesc`. Se ha mantenido la denominación simplificada en los pseudocódigos para no reproducir la cadena completa en cada referencia.

7.3 IMPLEMENTACIÓN

Se presentan primero los memorandos técnicos con las decisiones de diseño e implementación; a continuación, los pseudocódigos que las materializan.

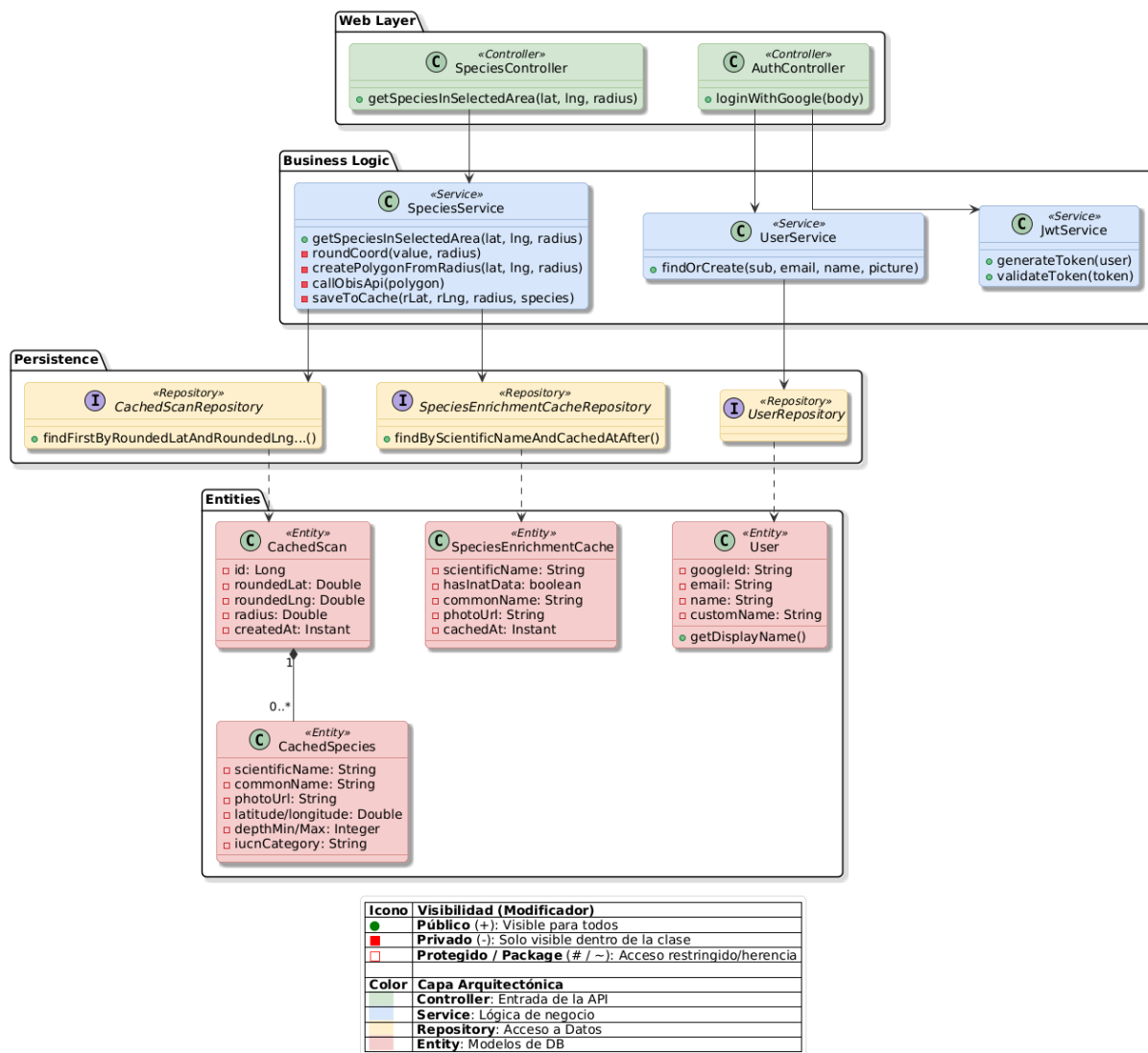


Figura 7.1: Diagrama UML de diseño para la iteración 1

Memorando técnico 001: Geometría de Búsqueda WKT	
Asunto	La API de OBIS no acepta búsqueda radial: requiere un polígono POLYGON((. . .)) en formato WKT.
Resumen	Generación de un polígono de 12 vértices aproximadamente circular desde coordenadas centrales y radio.
Factores causantes	El usuario define la zona como un punto central y un radio, pero la API de OBIS exige el área en formato WKT (polígono cerrado). Alternativamente, se podría haber usado un bounding box rectangular, no obstante sobreestimaría el área incluyendo registros fuera del radio real.
Solución	Se implementa <code>createPolygonFromRadius(lat, lng, radius)</code> , que aproxima el círculo con un polígono de 12 vértices equidistantes, corrigiendo la longitud por $\cos(\phi)$ para no deformar el área en latitudes altas. La cadena WKT resultante se codifica adecuadamente antes de insertarse en la URL de la petición HTTP.
Motivación	OBIS alberga >150M registros de biodiversidad marina validados científicamente. Es preferible adaptarse a su interfaz antes que buscar fuentes alternativas de menor calidad, ya que no existen muchas comparables.
Cuestiones abiertas	A latitudes altas (> 60) la aproximación esférica introduce error; para el caso de uso de Scubex (zonas costeras tropicales y templadas) es despreciable.
Alternativas	Bounding box: sencillo pero incluye esquinas fuera del radio real. Filtrado post-proceso: pedir área global y filtrar en memoria, descartado por transferencia de datos innecesaria.

Cuadro 7.4: Memorando técnico 001: Geometría WKT

Memorando técnico 002: Filtrado de Calidad Visual	
Asunto	OBIS devuelve microorganismos y especies sin fotografía, que carecen de valor para una app de buceo.
Resumen	Filtrado cruzado: se descartan especies sin entrada en iNaturalist. Las que tienen entrada pero carecen de fotografía se incluyen con un emoji representativo de su phylum como fallback visual en el frontend.
Factores causantes	(1) OBIS incluye todo tipo de registros biológicos marinos (bacterias, algas unicelulares, etc.) que no se ven al bucear. (2) Hay especies macroscópicas con nombre científico válido que no tienen observaciones fotografiadas en iNaturalist (ej. <i>The-nea muricata</i>).
Solución	Cada especie devuelta por OBIS se busca en iNaturalist. Si no aparece, se descarta: sin nombre común ni foto no aporta valor. Si aparece, se incluye siempre: si tiene foto se usa; si no, el frontend muestra un emoji según el phylum como fallback visual.
Motivación	Un buceador busca animales que pueda ver en el agua. Incluir microorganismos sería ruido.
Cuestiones abiertas	La latencia de N llamadas HTTP a iNaturalist (una por especie única) en el primer escaneo de una zona puede superar los 20 segundos. Resuelta en Memorando 004 mediante caché de enriquecimiento por especie.
Alternativas	Filtrado por phylum: descartar directamente los phyla microscópicos (bacterias, algas) sin consultar iNaturalist. Es más rápido, pero una especie macroscópica rara de un phylum “grande” pasaría el filtro aunque no tenga foto ni nombre común. Placeholder genérico: asignar siempre un emoji o imagen por grupo taxonómico sin consultar iNaturalist. Elimina las llamadas externas pero pierde la foto real y el nombre común cuando sí existen.

Cuadro 7.5: Memorando técnico 002: Filtrado de Calidad Visual

Memorando técnico 003: Caché L1 de Escaneos (CachedScan)	
Asunto	Latencia de 10–20 s por escaneo en zonas sin datos previos en caché.
Resumen	Caché de resultado completo por zona geográfica indexado por coordenadas redondeadas y radio, con TTL de 48 h.
Factores causantes	Un escaneo completo puede tardar entre 10 y 20 segundos. Repetir ese proceso cada vez que alguien consulta la misma zona es un desperdicio innecesario, tanto en tiempo de respuesta como en llamadas a APIs externas.
Solución	El resultado de cada escaneo se persiste en base de datos bajo una clave compuesta por coordenadas redondeadas y radio. El redondeo agrupa clicks cercanos en la misma clave, evitando duplicados por diferencias de centímetros. Si existe un escaneo de menos de 48 h para esa zona, se devuelve directamente sin tocar ninguna API externa.
Motivación	Capacidad de dar una respuesta instantánea a un escaneo, sin pasar por APIs. Plugins ya preconfigurados en base de datos, lo que hace la implementación mas simple
Cuestiones abiertas	Clicks en el umbral de redondeo (ej. 0.049° vs 0.051°) pueden generar un falso miss aunque las zonas sean prácticamente idénticas. Comportamiento aceptado por su baja frecuencia.
Alternativas	Redis: más rápido, pero añade servicio externo y coste. Spring Cache en memoria: no persiste entre reinicios del servidor (Railway reinicia en cada despliegue).

Cuadro 7.6: Memorando técnico 003: Caché L1 — CachedScan

Memorando técnico 004: Caché L2 de Enriquecimiento (SpeciesEnrichmentCache)

Asunto	Aun con el caché L1, los escaneos seguían siendo lentos debido a los <code>rateLimit</code> de iNaturalist.
Resumen	Se implementó un caché persistente por especie (con <code>scientificName</code> como clave única), que guarda los datos extra de iNaturalist y básicos de OBIS durante 30 días para ahorrar llamadas. También se implementó un semáforo de concurrencia, para limitar las llamadas en paralelo a iNaturalist.
Factores causantes	Pese a que 100 llamadas por minuto sean usualmente suficientes, a veces se quedaban cortas en zonas de alta biodiversidad. La caché L1 evita repetir escaneos de la misma zona, pero no ayuda cuando la misma especie aparece en zonas distintas.
Solución	La foto, el nombre común y los datos ecológicos de cada especie se guardan en base de datos usando el nombre científico como clave, con una validez de 30 días. La próxima vez que esa especie aparezca en cualquier escaneo, se recupera directamente de base de datos sin tocar ninguna API. Para no saturar iNaturalist, un semáforo limita a 3 las llamadas concurrentes de esta API mientras las demás esperan su turno. Si una especie no existe en iNaturalist, también se guarda esa ausencia, evitando repetir la consulta inútil durante 30 días.
Motivación	Los datos de iNaturalist y OBIS son siempre los mismos independientemente de dónde se encuentre. Y un TTL de 30 días equilibra frescura (actualizaciones de las especies) con eficiencia.
Cuestiones abiertas	Si el estado IUCN de una especie cambia dentro del período de TTL de 30 días, la caché servirá el dato desactualizado. Aceptado dado que dichos cambios son poco frecuentes.
Alternativas	Caché solo en L1: resuelve repeticiones en la misma zona, pero si la misma especie aparece en el Mediterráneo y en el Atlántico Norte, se vuelven a hacer todas las llamadas. Pre-carga batch: mantener un catálogo de especies actualizado es inviable dado que las zonas de buceo son heterogéneas y las especies que aparecen varían enormemente.

Cuadro 7.7: Memorando técnico 004: Caché L2 — SpeciesEnrichmentCache

Identificador	Descripción de la acción de alto nivel			
001	SpeciesService			
Métodos de alto nivel				
List<SpeciesResponse> getSpeciesInSelectedArea(lat:double, lng:double, radius:double)				
Pasos				
1. Se redondean las coordenadas recibidas según el radio con roundCoord (), para agrupar zonas cercanas en una misma clave de caché. A mayor radio, mayor es el redondeo				
2. Se consulta la base de datos buscando un escaneo previo de la misma zona (menos de 48h) con findBestMatch (). Si existe, se devuelve directamente sin llamar a ninguna API externa fin.				
3. Si no hay caché, se construye un polígono WKT de 12 vértices alrededor del punto central con createPolygonFromRadius () y se consultan los registros de OBIS con callObisApi ().				
4. Para cada especie única devuelta por OBIS, en paralelo:				
4.1. Se busca en la caché de enriquecimiento con findByNameAndCachedAtAfter (). Si ya existe y no tiene datos de iNaturalist, se descarta la especie.				
4.2. Si no está en caché, se consulta iNaturalist (máximo 3 llamadas simultáneas). Si no tiene foto o no aparece, se guarda como descartada para no volver a consultarla en 30 días.				
4.3. Si iNaturalist devuelve foto y nombre común, se obtienen datos ecológicos de OBIS (profundidad, temperatura, registros históricos, categoría IUCN) con callObisEcoStats () y se guarda todo en caché.				
5. Se persiste el resultado completo del escaneo con saveToCache () y se devuelve la lista de especies enriquecidas.				
Métodos de bajo nivel necesarios				
Paso	Clase	Método	Mem. Técn.	IU
1	SpeciesSvc.	double roundCoord(value, radius)	003	NO
3	ScanRepo.	Optional<CachedScan> findBestMatch(lat, lng, radius, cutoff)	003	NO
5	SpeciesSvc.	String createPolygonFromRadius(lat, lng, radius)	001	NO
6	SpeciesSvc.	List<ObisOccurrence> callObisApi(polygon)	001 002	NO
8.1	EnrichRepo.	Optional<SpeciesEnrichmentCache> findByNameAndCachedAtAfter(name, after)	002 004	NO
8.4- 8.6	SpeciesSvc.	ObisEcoData callObisEcoStats(name, executor)	004	NO
9	SpeciesSvc.	void saveToCache(rLat, rLng, radius, species)	003	NO
			004	

47

Memorando técnico 005: Autenticación OAuth2 + JWT	
Asunto	La aplicación necesita identificar usuarios sin gestionar contraseñas ni sesiones de servidor.
Resumen	Validación del ID token de Google mediante su API de tokeninfo y emisión de JWT firmado con HMAC-SHA256, TTL 7 días.
Factores causantes	Scubex no tiene servidor de sesiones, así que la identidad del usuario tiene que viajar en cada petición. Implementar registro propio con contraseñas añadiría complejidad sin aportar nada al usuario.
Solución	El usuario hace click en “Continuar con Google”; Google devuelve un token que el backend valida directamente contra su API. Si es válido y el campo aud corresponde a la app (evitando suplantación), se crea o recupera el usuario y se emite un JWT propio firmado con HMAC-SHA256 que el cliente usará en todas sus peticiones autenticadas.
Motivación	Sin sesiones de servidor: compatible con despliegue stateless en Railway. Sin contraseñas: menor superficie de ataque. Google verifica la identidad: Scubex no almacena credenciales sensibles.
Cuestiones abiertas	Sin <i>refresh tokens</i> : tras 7 días el usuario debe volver a autenticarse. Sin revocación de tokens individuales (logout invalida solo la copia en localStorage).
Alternativas	Auth0 / Firebase Authentication : delega toda la lógica de autenticación a un servicio externo, simplificando la implementación, pero introduce dependencia y coste si el uso escala. Session cookies HttpOnly : Requieren que el servidor mantenga estado de sesión, incompatible con el despliegue stateless en Railway.

Cuadro 7.8: Memorando técnico 005: Autenticación OAuth2 + JWT

Identificador	Descripción de la acción de alto nivel			
002	AuthController			
Métodos de alto nivel				
ResponseEntity<><?> loginWithGoogle(body:Map<String,String>)				
Pasos				
1. Se extrae el token de Google del cuerpo de la petición. Si está vacío o no se ha enviado, se rechaza con 400 Bad Request.				
2. Se valida el token contactando a Google con getObject(tokeninfo?id_token=...). Si Google no responde o el token es inválido, se rechaza con 401 Unauthorized.				
3. Se verifica que el campo aud de la respuesta coincide con el G00GLE_CLIENT_ID de la aplicación, para evitar que tokens emitidos para otras apps sean aceptados.				
4. Con los datos de identidad confirmados (sub, email, nombre, foto), se busca o crea el usuario en la base de datos con findOrCreate(). Si el usuario ya existe, se recupera; si es nuevo, se registra.				
5. Se genera un JWT firmado con HMAC-SHA256 mediante generateToken() y se devuelve al cliente junto con los datos básicos del perfil. A partir de aquí el cliente usa este JWT en todas sus peticiones autenticadas.				
Métodos de bajo nivel necesarios				
Paso	Clase	Método	Mem. Técn.	IU
3	RestTemplate	Map<String,String> getObject(url, Map.class)	005	NO
6	UserSvc.	User findOrCreate(sub, email, name, picture)	005	SI
7	JwtSvc.	String generateToken(user)	005	NO

7.4 PRUEBAS

Se han implementado las siguientes pruebas empleando JUnit5 y Mockito para aislar las dependencias externas (OBIS, iNaturalist, Google).

7.4.1 SpeciesServiceTest

- Generación de polígono WKT válido (shouldGenerateValidWKTPolygon): dado el punto (36.5, -4.0) y radio 5000 m, verifica que la URI generada contenga geometry=POLYGON((...)) geométricamente cerrado (primer y último punto idénticos) con al menos 4 vértices.

- Filtrado de microorganismos sin resultados (`shouldFilterMicroorganismsWithZeroResults`): OBIS devuelve *Pycnococcaceae*; iNaturalist responde `total_results=0`. Verifica que `getSpeciesInSelectedArea` devuelve lista vacía y que se han invocado ambas APIs.
- Enriquecimiento de especie con foto (`shouldEnrichSpeciesWithPhoto`): OBIS devuelve *Chimaera monstrosa*; iNaturalist devuelve `preferred_common_name = -abbit Fish` y `photoUrl` de S3. Verifica que `SpeciesResponse.commonName == Rabbit Fish` y `photoUrl != null`.
- Tolerancia a timeout de iNaturalist (`shouldHandleINaturalistTimeout`): OBIS devuelve dos especies; la primera llamada a iNaturalist lanza `RestClientException`; la segunda funciona. Verifica que el resultado contiene exactamente una especie y que sus campos son no nulos (resiliencia ante fallos parciales).
- Estadísticas ecológicas: profundidad, temperatura e IUCN (`shouldFetchEcoStatsAndPopulate`): OBIS devuelve *Octopus vulgaris* con entrada válida en iNaturalist. Los tres endpoints de eco-estadísticas de OBIS (`/statistics`, `/statistics/env` y `/checklist/redlist`) se mockean con datos reales: 1500 registros globales, rango de años 1980–2024, profundidad 5–40 m, temperatura 18–26 °C y categoría IUCN LC. Verifica que todos esos campos llegan correctamente en el `SpeciesResponse` final, cubriendo los tres lambdas de `callObisEcoStats()` que los tests anteriores no alcanzaban.

7.4.2 JwtServiceTest

- Generación de token con claims correctos (`generateToken_containsExpectedClaims`): dado un `User` con `googleId="google-123"`, verifica que el token generado contiene `subject=googleId`, `email`, `name` y `picture` correctos.
- Validación de token: devuelve subject (`validateToken_returnsSubject`): token válido devuelve `googleId` como subject.
- Validación de token manipulado: lanza excepción (`validateToken_tamperedToken_throwsException`): token con firma modificada lanza excepción al validar.

7.4.3 AuthControllerTest

- Login sin credencial: devuelve 400 (`login_missingCredential_returns400`): petición `POST /api/auth/google` sin campo `credential` devuelve 400 `Bad Re-`

quest.

- Login con token rechazado por Google: devuelve 401 (`login_googleRejectsToken_returns401`). `restTemplate.getForObject` lanza `RuntimeException`. Verifica que el controlador devuelve 401 `Unauthorized`.
- Login válido: devuelve 200 con token JWT (`login_validCredential_returns200WithToken`): `googleInfo` mockado con `aud=CLIENT_ID`; `userService.findOrCreate` devuelve usuario; `jwtService.generateToken` devuelve `"mocked.jwt.token"`. Verifica respuesta 200 OK con token y `user.email` correctos.

7.5 DESPLIEGUE

Al cierre de esta iteración la aplicación quedó desplegada en la infraestructura descrita en el Arranque del proyecto:

- **Backend (Railway):** el proceso de arranque ejecuta `mvn package` y lanza el JAR resultante. Las variables de entorno `GOOGLE_CLIENT_ID`, `JWT_SECRET`, `DATABASE_URL` y `CORS_ALLOWED_ORIGINS` se configuran desde el panel de Railway.
- **Frontend (Vercel):** cada push a `main` desencadena un despliegue automático. La variable `VITE_API_BASE_URL` apunta a la URL pública del backend en Railway y `VITE_MAPTILER_KEY` provee acceso a las teselas marineras de MapTiler.
- **Base de datos (PostgreSQL en Railway):** accedida mediante `DATABASE_URL`; Hibernate gestiona el esquema automáticamente (`ddl-auto=update`).

La aplicación es accesible públicamente en `https://scubex.vercel.app` desde la finalización de esta iteración.

ITERACIÓN 2: SISTEMA CLIMÁTICO

*E*sta iteración añade la segunda parte del MVP de Scubex: El panel de condiciones meteorológicas y marinas, el usuario puede consultar en tiempo real si una zona es apta o no para bucear antes de meterse al agua.

8.1 CARACTERÍSTICAS DESARROLLADAS ESTA ITERACIÓN

1. Panel de clima con datos atmosféricos y marinos en tiempo real, evaluación automática de condiciones de buceo y caché de 30 minutos. Ver Tabla §8.1.

Análisis de valor aportado: Panel de Clima	
Propuesta	Consulta del estado del mar y la atmósfera para el punto marcado en el mapa, con evaluación automática de si las condiciones son aptas para bucear.
Valor	Un buceador no puede planificar una inmersión sin saber si el mar está en calma. El panel proporciona en un único vistazo los datos que determinan la viabilidad de la salida olas, corriente, visibilidad y tiempo sin que el usuario tenga que consultar varias fuentes ni interpretar valores crudos.
Coste	Integración con dos endpoints de Open-Meteo (atmospheric forecast y marine API), ambos gratuitos y sin clave. El único esfuerzo no trivial es definir los criterios de evaluación: qué umbrales y pesos determinan si una condición es buena, tolerable o adversa desde el punto de vista del buceo.
Opciones	Windy: mapas muy visuales de viento y oleaje, pero no tiene API pública gratuita para datos en crudo. Evaluación manual: mostrar solo los valores numéricos y dejar que el usuario decida; eliminaba la lógica del servidor pero obligaba al usuario a conocer los umbrales de seguridad.
Riesgos	Open-Meteo no ofrece SLA; una caída devuelve error sin datos de clima. Los datos marinos no existen en zonas continentales o bahías cerradas, por lo que esos campos pueden llegar vacíos. La evaluación automática no distingue condiciones locales como resacas puntuales o fondos con corrientes de fondo intensas.
Deuda técnica	Solo se muestran condiciones en el instante actual; no hay previsión por horas ni por días. Los umbrales de evaluación son globales e iguales para todos los usuarios, sin distinción entre perfiles (principiante vs. avanzado).

Cuadro 8.1: Análisis de valor aportado: Panel de Clima

8.2 DISEÑO

El diagrama UML de la Fig. §8.1 muestra las relaciones entre los componentes del sistema climático: controlador, servicio, entidad de caché y los dos clientes de Open-Meteo.

8.3 IMPLEMENTACIÓN

Se presentan primero los memorandos técnicos con las decisiones de diseño; a continuación, el pseudocódigo que las materializa.

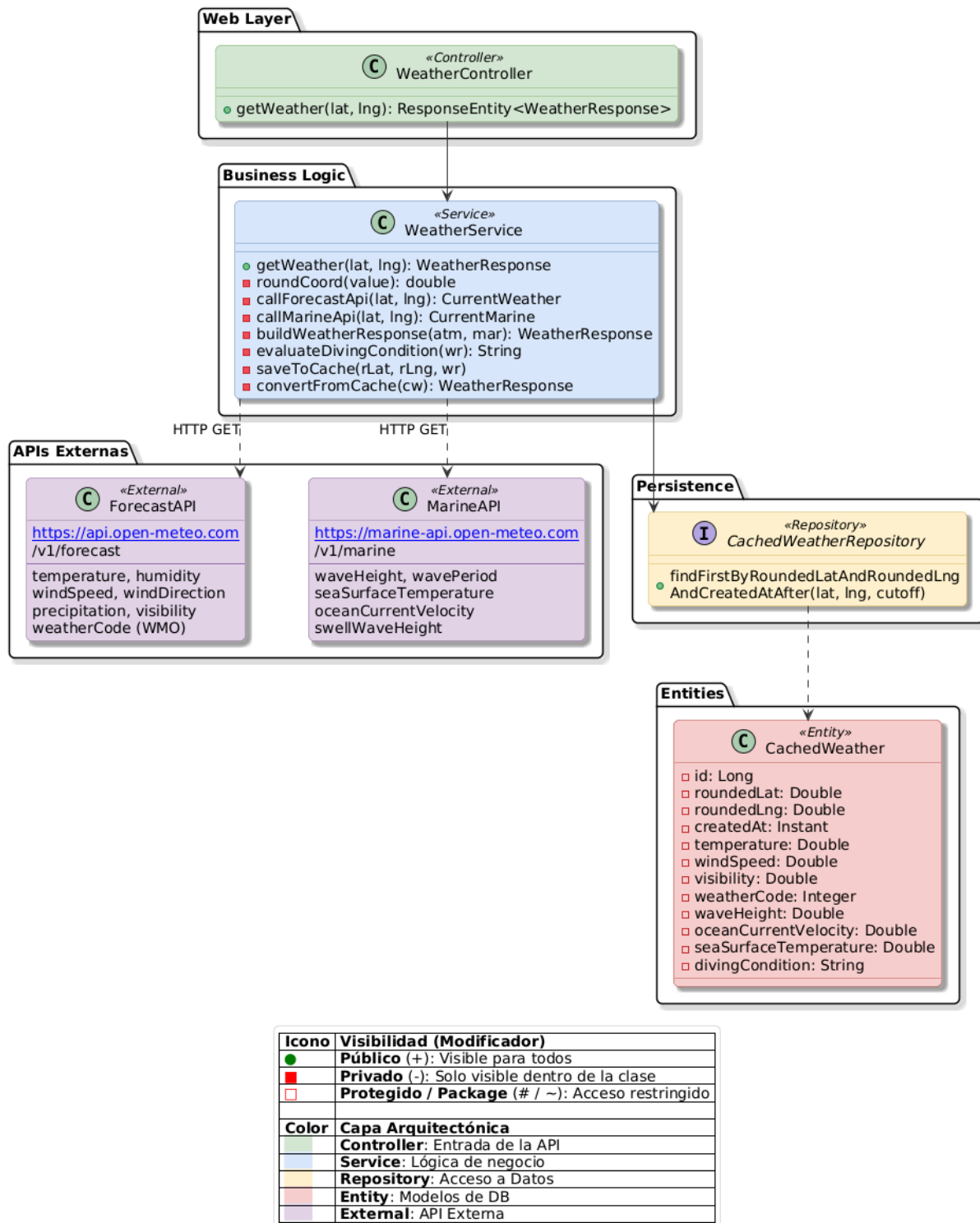


Figura 8.1: Diagrama UML de diseño para la iteración 2

Memorando técnico 006: WeatherResponse — Datos de Dos Fuentes	
Asunto	Los datos atmosféricos (temperatura, viento, lluvia) y los datos marinos (olas, corrientes, temperatura del agua) provienen de dos endpoints distintos de Open-Meteo.
Resumen	Se realizan dos llamadas independientes y sus resultados se fusionan en un único objeto WeatherResponse que el frontend consume en una sola petición.
Factores causantes	Open-Meteo separa su API en dos servicios: el de previsión atmosférica (<code>api.open-meteo.com</code>) y el marino (<code>marine-api.open-meteo.com</code>). El panel de Scubex necesita datos de ambos para ser útil.
Solución	WeatherService llama de forma independiente a <code>callForecastApi()</code> y <code>callMarineApi()</code> , y combina los resultados con <code>buildWeatherResponse()</code> . Si una de las dos APIs falla o devuelve nulo (por ejemplo, la marina en zonas continentales), los campos correspondientes quedan vacíos en la respuesta pero el resto de datos se sigue devolviendo correctamente.
Motivación	Principalmente la riqueza de los datos, tener datos atmosféricos y marinos es la única manera que la implementación de datos climáticos pueda ser realmente útil. Además separar las dos llamadas permite tolerancia a fallos parciales: si la API marina no tiene datos para una zona interior, el panel sigue mostrando temperatura, viento y lluvia, que sí están disponibles en cualquier punto del planeta.
Cuestiones abiertas	Las dos llamadas se realizan de forma secuencial. Paralelizarlas con <code>CompletableFuture</code> reduciría la latencia en los casos sin caché.
Alternativas	Windy Meteo API: requiere suscripción de pago para acceso a datos en crudo.

Cuadro 8.2: Memorando técnico 006: WeatherResponse

Memorando técnico 007: Evaluación Automática de Condiciones de Buceo	
Asunto	El usuario necesita saber si puede bucear sin tener que interpretar él mismo una tabla de valores meteorológicos.
Resumen	Un algoritmo de puntuación ponderada clasifica las condiciones en <i>buenas</i> , <i>tolerables</i> o <i>adversas</i> , con anulaciones directas ante valores extremos en los factores más críticos.
Factores causantes	Un buceador no pide saber exactamente que hay olas de 1,3 m; quiere saber si puede entrar al agua. Presentar solo los valores crudos obliga al usuario a conocer umbrales de seguridad que varían según su experiencia y el tipo de buceo.
Solución	<code>evaluateDivingCondition()</code> aplica primero un conjunto de anulaciones críticas: oleaje > 2,5 m, corriente > 5 km/h, viento > 40 km/h, tormenta eléctrica (código WMO 95/96/99) o visibilidad < 1 000 m devuelven inmediatamente "bad". Si no se cumple ninguna anulación, se puntúan seis factores con pesos distintos (oleaje = 3, corriente = 3, visibilidad = 2, viento = 2, código meteorológico = 2, probabilidad de lluvia = 1) y se calcula la media ponderada: $\geq 1,5$ es <i>buenas</i> , $\geq 0,8$ es <i>tolerables</i> , resto <i>adversas</i> .
Motivación	El oleaje y la corriente son los factores más peligrosos para un buceador; por eso tienen el peso más alto y los umbrales de anulación más conservadores. La probabilidad de lluvia tiene el peso mínimo porque no afecta directamente a la seguridad bajo el agua.
Cuestiones abiertas	Los umbrales son fijos e iguales para todos los usuarios. Un perfil de "buceador avanzado" podría tolerar corrientes de hasta 4 km/h sin problema, mientras que un principiante debería ver <i>adversas</i> con corriente de 2 km/h.
Alternativas	Umbrales fijos sin ponderación: más simple, pero no tiene una correspondencia correcta con los factores del buceo, el oleaje siempre importará mas que si va a llover.

Cuadro 8.3: Memorando técnico 007: Evaluación de Condiciones

Memorando técnico 008: Caché de Clima (CachedWeather)	
Asunto	Cada vez que el usuario mueve el mapa o cambia el punto, se lanzarían dos llamadas a Open-Meteo, añadiendo latencia innecesaria.
Resumen	Los datos de clima se guardan en base de datos durante 30 minutos por coordenadas redondeadas, evitando repetir las llamadas para puntos cercanos o consultas frecuentes sobre la misma zona.
Factores causantes	El mapa es interactivo y el usuario puede mover el punto libremente. Sin caché, cada pequeño desplazamiento generaría dos peticiones HTTP hacia Open-Meteo, con la latencia acumulada correspondiente.
Solución	Se aplica el mismo esquema de redondeo de coordenadas descrito en el Memorando §7.6: las coordenadas se redondean antes de usarlas como clave de búsqueda, agrupando puntos cercanos bajo la misma entrada. Si existe un registro de menos de 30 minutos para esas coordenadas en la tabla <code>cached_weather</code> , se devuelve directamente sin tocar Open-Meteo. El TTL de 30 minutos equilibra frescura de los datos climáticos con eficiencia: el tiempo meteorológico no cambia de forma significativa en ventanas menores.
Motivación	Los datos de clima son mucho más volátiles que los de biodiversidad, por lo que el TTL es 30 minutos frente a las 48 horas del caché de escaneos (Memorando §7.6). Reutilizar el mismo mecanismo de caché en base de datos evita añadir nuevos servicios: la tabla <code>cached_weather</code> sigue exactamente el mismo patrón que <code>CachedScan</code> .
Cuestiones abiertas	El caso límite del redondeo (dos puntos muy próximos que caen en celdas distintas) es el mismo ya reconocido en el Memorando §7.6 para el caché de escaneos, y se acepta por la misma razón: baja frecuencia e impacto despreciable.
Alternativas	TTL más corto (5–10 min): más fresco, pero aumenta significativamente las llamadas a Open-Meteo. Sin caché: máxima frescura, pero penaliza la experiencia en sesiones de exploración del mapa.

Cuadro 8.4: Memorando técnico 008: Caché de Clima

Identificador	Descripción de la acción de alto nivel			
003	WeatherService			
Métodos de alto nivel				
WeatherResponse getWeather(lat:double, lng:double)				
Pasos				
1. Se redondean las coordenadas a 2 decimales para agrupar puntos cercanos en la misma clave de caché.				
2. Se busca en base de datos un resultado reciente (menos de 30 min) para esas coordenadas. Si existe, se convierte al DTO de respuesta y se devuelve directamente, fin.				
3. Si no hay caché, se consulta la API de previsión atmosférica con callForecastApi(): temperatura, humedad, viento, precipitación, visibilidad y código meteorológico WMO.				
4. Se consulta la API marina con callMarineApi(): altura de ola, periodo, corriente oceánica, temperatura superficial del mar y oleaje de fondo. Si la zona no tiene datos marinos (zona continental), esta llamada devuelve nulo y esos campos quedan vacíos.				
5. Se fusionan los datos de ambas APIs en un WeatherResponse con buildWeatherResponse().				
6. Se evalúan las condiciones de buceo con evaluateDivingCondition() y se añade el resultado ("good", "moderate" o "bad") al WeatherResponse.				
7. Se guarda el resultado completo en caché con saveToCache() y se devuelve al cliente.				
Métodos de bajo nivel necesarios				
Paso	Clase	Método	Mem. Téc.	IU
1	WeatherSvc	double roundCoord(value)	008	NO
2	WeatherRepo	Optional<CachedWeather> findFirstByRoundedLatAndRoundedLngAndCreatedAtAfter(lat, lng, cutoff)	008	NO
3	WeatherSvc	ForecastApiResponse.CurrentWeather callForecastApi(lat, lng)	006	NO
4	WeatherSvc	MarineApiResponse.CurrentMarine callMarineApi(lat, lng)	006	NO
5	WeatherSvc	WeatherResponse buildWeatherResponse(atmosphere, marine)	006	SI
6	WeatherSvc	String evaluateDivingCondition(wr)	007	SI
7	WeatherSvc	void saveToCache(roundedLat, roundedLng, wr)	008	NO

8.4 PRUEBAS

Las pruebas unitarias de `WeatherService` se han implementado en `WeatherServiceTest`, siguiendo el mismo patrón basado en Mockito que el resto del proyecto. Los cuatro casos de prueba son los siguientes:

- **Cache hit** (`cacheHit_returnsCachedDataWithoutCallingExternalApis`): se inyecta en el repositorio un `CachedWeather` con antigüedad de 10 minutos. El servicio devuelve los campos cacheados y no invoca a `RestTemplate` en ningún momento (`verifyNoInteractions`).
- **Evaluación adversa por anulación crítica** (`criticalOverride_waveHeightExceeds2_5m_returnsBadDivingCondition`): la API marina devuelve oleaje de 3,0m con condiciones atmosféricas ideales. Se verifica que `divingCondition` es "bad", lo que confirma que la anulación crítica se dispara antes de calcular la puntuación ponderada.
- **Evaluación correcta sin datos marinos** (`noMarineData_continentalArea_computesConditionWithoutMarineData`): la API marina devuelve cuerpo nulo (`ResponseEntity` con `null`). Se verifica que no se lanza excepción, que los campos marinos son nulos y que `divingCondition` no es nulo, es decir, el algoritmo evalúa correctamente con solo factores atmosféricos.
- **Tolerancia a fallo de API** (`forecastApiFailure_returnsNullAtmosphericFieldsWithoutThrowingException`): `restTemplate.getForEntity` lanza `RestClientException` para la URL atmosférica. Se verifica que el servicio no propaga la excepción, que los campos atmosféricos son nulos y que los datos marinos siguen presentes en la respuesta.

8.5 DESPLIEGUE

Esta iteración no requirió cambios en la infraestructura existente. Open-Meteo es una API pública sin autenticación, por lo que no fue necesario añadir nuevas variables de entorno ni credenciales. La tabla `cached_weather` fue creada automáticamente por Hibernate al arrancar el servicio en Railway con `ddl-auto=update`.

Las URLs de los dos endpoints de Open-Meteo se externalizan en `application.properties` mediante las propiedades `open-meteo.weather.url` y `open-meteo.marine.url`, lo que permite cambiarlas sin recompilar la aplicación.

ITERACIÓN 3: PUBLICACIONES

***E**sta iteración añade el final del MVP de Scubex: Un sistema de publicaciones. Los usuarios autenticados pueden crear publicaciones geolocalizadas con imagen desde cualquier punto del mapa, y cualquier visitante puede consultarlas directamente desde los marcadores del mapa.*

9.1 CARACTERÍSTICAS DESARROLLADAS ESTA ITERACIÓN

1. Publicaciones geolocalizadas: crear, editar y eliminar entradas con título, descripción e imagen, ancladas a un punto del mapa. Ver Tabla §9.1.

Análisis de valor aportado: Publicaciones Geolocalizadas	
Propuesta	Un sistema de publicaciones integrado en el mapa: el usuario hace clic en cualquier punto, escribe un título, una descripción y adjunta una foto, y la publicación queda anclada a esas coordenadas. Cualquier persona puede verlas explorando el mapa.
Valor	Convierte Scubex en un diario colectivo de inmersiones. Un buceador puede compartir qué encontró en una cala concreta, y otros usuarios descubren ese contenido simplemente explorando la zona en el mapa, sin necesidad de buscar por texto ni conocer el nombre del lugar.
Coste	Diseñar el modelo de datos con coordenadas indexadas, gestionar el almacenamiento de imágenes en base de datos, implementar el filtrado por área visible y resolver el nombre del lugar a partir de las coordenadas en el frontend.
Opciones	Feed cronológico sin mapa: más simple de implementar, pero pierde la dimensión geográfica que diferencia Scubex de cualquier otra red social. Spots predefinidos: el usuario solo puede publicar en zonas conocidas de antemano, lo que elimina la libertad de compartir descubrimientos propios.
Riesgos	Si el usuario hace zoom muy hacia fuera, el área visible es tan grande que la respuesta podría incluir un volumen muy alto de publicaciones. Tampoco hay ningún control sobre dónde se publica: un usuario puede anclar una publicación en tierra firme o en cualquier punto sin relación con el buceo.
Deuda técnica	Sin límite de área en la consulta. Sin moderación de contenido ni restricción geográfica.

Cuadro 9.1: Análisis de valor aportado: Publicaciones Geolocalizadas

9.2 DISEÑO

El diagrama UML de la Fig. §9.1 muestra los componentes del sistema de publicaciones: controladores, servicio, repositorios y entidades, así como la dependencia del módulo de geocoding en el frontend.

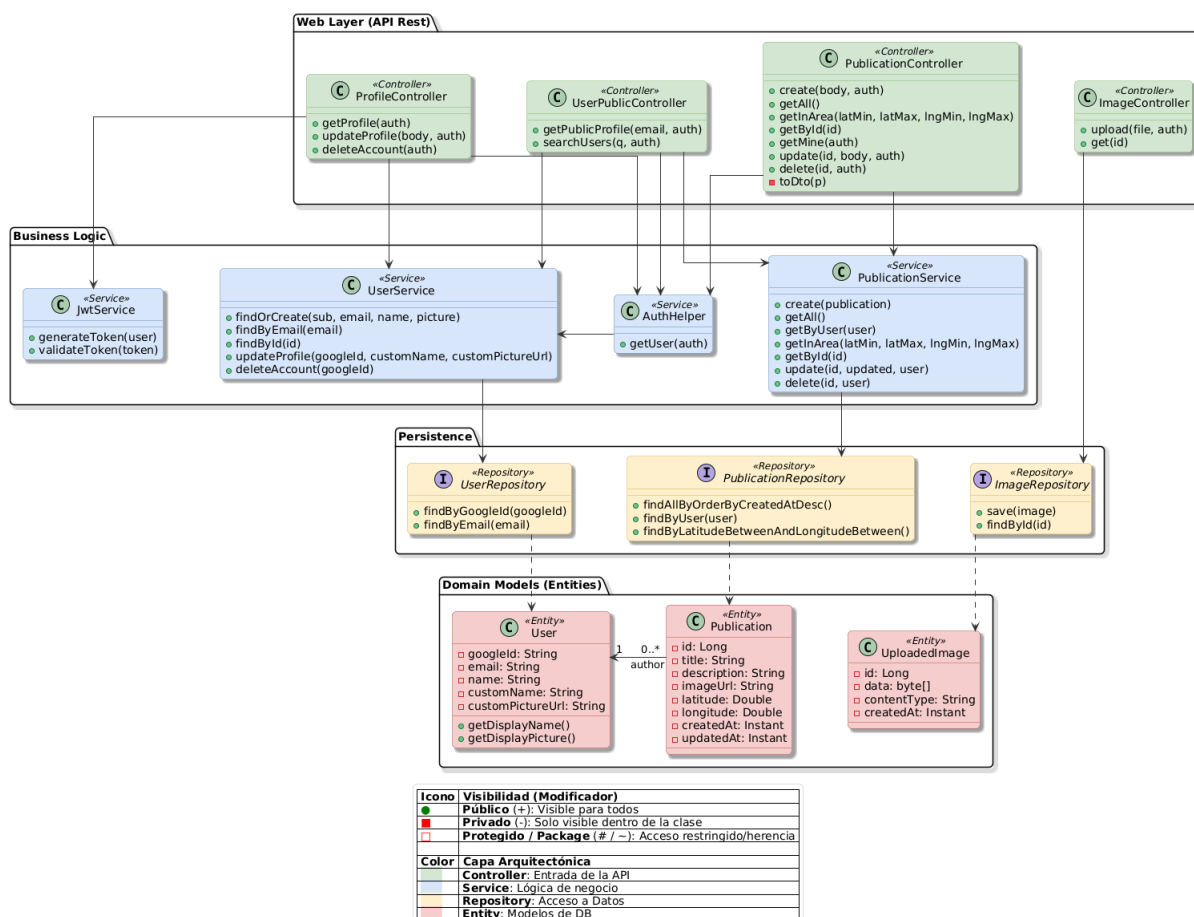


Figura 9.1: Diagrama UML de diseño para la iteración 3

9.3 IMPLEMENTACIÓN

Se presentan primero los memorandos técnicos con las decisiones de diseño; a continuación, los pseudocódigos que las materializan.

Nota: El memorando 10 no tiene pseudocódigo de backend propio ya que la lógica es íntegramente del frontend; da soporte al flujo de creación de publicaciones, donde el nombre del lugar se muestra al usuario antes de confirmar.

Memorando técnico 009: Filtrado de Publicaciones por Viewport del Mapa	
Asunto	Descargar todas las publicaciones existentes en cada consulta del mapa haría la aplicación cada vez más lenta conforme crece el contenido.
Resumen	El servidor devuelve únicamente las publicaciones que caen dentro del área visible del mapa en cada momento, apoyándose en un índice de coordenadas para que la consulta sea eficiente.
Factores causantes	Si cada vez que se abre el mapa o se mueve se descargaran todas las publicaciones de la base de datos, la respuesta crecería indefinidamente con el tiempo. En zonas con mucho contenido, esto haría la experiencia lenta incluso para usuarios con buena conexión.
Solución	El frontend envía al servidor los límites del área visible (latitud y longitud mínima y máxima) con cada consulta. El servidor filtra y devuelve únicamente las publicaciones que caen dentro de ese rectángulo. Para que esta búsqueda sea eficiente sin importar cuántas publicaciones haya, las coordenadas de cada publicación tienen un índice en base de datos, de modo que la consulta no tiene que recorrer la tabla entera.
Motivación	Es la forma natural de escalar un sistema con contenido georreferenciado: el usuario no necesita lo que no puede ver en pantalla. El índice en coordenadas garantiza que añadir publicaciones no penalice el rendimiento de la consulta.
Cuestiones abiertas	Si el usuario hace zoom muy hacia fuera, los límites del mapa abarcan un área enorme y la respuesta podría incluir un número muy alto de publicaciones de golpe.
Alternativas	Agrupación geográfica: reduce la carga visual de marcadores en el mapa, pero no reduce la cantidad de datos transferidos.

Cuadro 9.2: Memorando técnico 009: Filtrado por Viewport del Mapa

Memorando técnico 010: Geocoding Inverso con Nominatim	
Asunto	Las publicaciones se anclan a coordenadas, pero mostrar latitud y longitud crudas no aporta contexto al usuario.
Resumen	El frontend traduce las coordenadas de cada publicación a un nombre de lugar legible consultando Nominatim, y guarda los resultados en memoria para no repetir la consulta durante la misma sesión.
Factores causantes	Tanto el formulario de creación como la vista de detalle muestran dónde está el punto marcado. Para un usuario es mucho más informativo leer “Punta de la Mona, Granada” que ver un par de números decimales.
Solución	Cuando se necesita el nombre de un punto, se consulta Nominatim probando primero el nivel de detalle más alto (playas, puntos de interés), luego el intermedio (ciudad o municipio) y finalmente el regional. Si ningún nivel devuelve resultado, se muestra “Mar abierto” como alternativa. Para no repetir la misma consulta si el usuario vuelve al mismo punto, los resultados se guardan en memoria durante toda la sesión.
Motivación	Nominatim es gratuito y no requiere ninguna clave de API, lo que lo convierte en la opción más directa. Probar niveles de detalle de mayor a menor garantiza el nombre más concreto posible sin lanzar varias peticiones en paralelo. Guardar los resultados en memoria es suficiente: si el usuario cierra la pestaña, la caché se limpia sola y en la siguiente sesión los datos de Nominatim seguirán siendo los mismos.
Cuestiones abiertas	Nominatim limita las peticiones a una por segundo para uso no comercial. En sesiones de exploración rápida del mapa, las peticiones de puntos que el usuario abandona antes de que se resuelvan se cancelan automáticamente, lo que ayuda a no saturar el servicio; pero sin una cola controlada podría alcanzarse el límite en casos extremos.
Alternativas	Google Maps Geocoding API: mayor precisión, pero de pago.

Cuadro 9.3: Memorando técnico 010: Geocoding Inverso con Nominatim

Identificador	Descripción de la acción de alto nivel			
004	PublicationController			
Métodos de alto nivel				
ResponseEntity<?> create(body:Map<String,Object>, auth:Authentication)				
Pasos				
1. Se comprueba que el usuario tiene sesión activa con authHelper.getUser(). Si no la tiene, se devuelve 401 Unauthorized.				
2. Se comprueba que el cuerpo de la petición incluye un título y unas coordenadas. Si alguno falta o el título está vacío, se devuelve 400 Bad Request.				
3. Se construye la publicación con el título, la descripción, la URL de imagen, las coordenadas y el usuario autenticado como autor.				
4. Se guarda la publicación en base de datos con publicationService.create(), que delega en publicationRepository.save(). La fecha de creación se asigna automáticamente en el momento de la inserción.				
5. Se convierte la publicación guardada a un objeto de respuesta con toDto(), incluyendo los datos del autor y los contadores de interacciones, y se devuelve al cliente.				
Métodos de bajo nivel necesarios				
Paso	Clase	Método	Mem. Técn.	IU
1	AuthHelper	User getUser(auth)	005	NO
4	PubRepo.	Publication save(publication)	009	NO
5	PubController	Map<String,Object> toDto(p)	009	SI

Memorando técnico 011: Almacenamiento de Imágenes como BLOB	
Asunto	Las publicaciones incluyen una fotografía adjunta que debe conservarse de forma permanente.
Resumen	Las imágenes se guardan como BLOB directamente en PostgreSQL. El servidor las sirve al navegador con una cabecera de caché de un año, de modo que una vez descargadas no se vuelven a pedir.
Factores causantes	Los contenedores de Railway no tienen almacenamiento en disco persistente: cualquier fichero guardado localmente desaparece al reiniciar el servidor. Guardar las imágenes en la propia base de datos resuelve el problema sin añadir ningún servicio externo.
Solución	Al subir una fotografía, el servidor comprueba que el formato sea aceptado (JPEG, PNG, GIF o WebP) y que no supere los 5 MB. Si pasa esa validación, los datos del fichero se guardan en la base de datos junto al tipo de formato. Para recuperarla, el servidor la lee de la base de datos y la envía al navegador indicándole que la almacene en caché durante un año, ya que una imagen identificada por su identificador nunca cambia.
Motivación	PostgreSQL ya forma parte del proyecto, así que no se necesita ningún servicio adicional ni credenciales extra. Para el volumen de imágenes esperado en Scubex la solución es perfectamente suficiente y evita introducir dependencias externas que habría que mantener.
Cuestiones abiertas	Las imágenes se sirven siempre al tamaño original, sin reducción ni compresión. No existe un límite de imágenes por usuario.
Alternativas	Amazon S3: más escalable a largo plazo, pero introduce coste y complejidad innecesarios para el alcance actual.

Cuadro 9.4: Memorando técnico 011: Almacenamiento de Imágenes como BLOB

Identificador	Descripción de la acción de alto nivel			
005	ImageController			
Métodos de alto nivel				
ResponseEntity<?> upload(file:MultipartFile, auth:Authentication)				
Pasos				
1. Se comprueba que el usuario tiene sesión activa. Si no la tiene, se devuelve 401 Unauthorized.				
2. Se valida el archivo: no puede estar vacío, no puede superar los 5 MB y el formato debe ser JPEG, PNG, GIF o WebP. Si no cumple alguna de estas condiciones, se devuelve 400 Bad Request.				
3. Se leen los datos del archivo con file.getBytes() y se prepara la entidad de imagen con el contenido binario y el tipo de formato.				
4. Se guarda la imagen en base de datos con imageRepository.save(). La fecha de subida se asigna automáticamente en el momento de la inserción.				
5. Se devuelve al cliente la URL que identifica la imagen subida, lista para usarse como referencia al crear o editar la publicación.				
Métodos de bajo nivel necesarios				
Paso	Clase	Método	Mem. Téc.	IU
3	MultipartFile	byte[] getBytes()	011	NO
4	ImageRepo.	UploadedImage save(image)	011	NO

9.4 PRUEBAS

Las pruebas unitarias de esta iteración se reparten en dos suites: `PublicationServiceTest` e `ImageControllerTest`, siguiendo el mismo patrón basado en Mockito y MockMvc que el resto del proyecto.

`PublicationServiceTest` cubre tres casos:

- **Filtrado por área** (`getInArea_returnsOnlyPublicationsInsideBoundingBox`): el repositorio se configura para devolver dos publicaciones concretas al recibir un bounding box; se verifica que el servicio retorna exactamente esas dos y que el repositorio fue invocado con los límites correctos.
- **Edición por propietario ajeno** (`update_byNonOwner_returnsNull`): se intenta editar una publicación con un usuario distinto al propietario; se verifica que el

resultado es nulo y que `publicationRepository.save()` no fue llamado.

- **Eliminación por propietario ajeno** (`delete_byNonOwner_returnsFalse`): se intenta borrar una publicación con un usuario distinto al propietario; se verifica que el resultado es `false` y que `publicationRepository.delete()` no fue invocado.

`ImageControllerTest` cubre ocho casos:

- **Subida sin autenticación** (`upload_unauthenticated_returns401`): llamar al endpoint sin sesión activa devuelve 401 sin persistir nada.
- **Archivo vacío** (`upload_emptyFile_returns400`): un archivo de cero bytes devuelve 400 sin persistir nada.
- **Archivo demasiado grande** (`upload_fileTooLarge_returns400`): un archivo superior a 5 MB devuelve 400 sin persistir nada.
- **Content-Type nulo** (`upload_nullContentType_returns400`): un archivo sin tipo de contenido declarado devuelve 400 sin persistir nada.
- **Formato no permitido** (`upload_invalidMimeType_returns400`): un archivo con Content-Type distinto de JPEG, PNG, GIF o WebP devuelve 400 sin persistir nada.
- **Subida válida** (`upload_validFile_returns200WithImageUrl`): un JPEG válido con usuario autenticado devuelve 200 con la `imageUrl` correcta, y se verifica que `imageRepository.save()` fue invocado exactamente una vez.
- **Descarga de imagen existente** (`get_existingImage_returns200WithBytes`): solicitar una imagen existente devuelve 200, el Content-Type correcto y los bytes de la imagen.
- **Descarga de imagen inexistente** (`get_nonExistingImage_returns404`): solicitar un identificador que no existe en base de datos devuelve 404.

9.5 DESPLIEGUE

Esta iteración añadió dos nuevas tablas a la base de datos: `publications` y `uploaded_images`, ambas creadas automáticamente por Hibernate al arrancar el servicio en Railway con `ddl-auto=update`. No fue necesario añadir nuevas variables de entorno

ni credenciales, ya que el almacenamiento de imágenes se realiza en la misma base de datos PostgreSQL existente.

ITERACIÓN 4: INTERACCIONES Y RED SOCIAL

*E*sta iteración convierte Scubex en una red social de buceo. Se añaden likes, guardados y comentarios con menciones en las publicaciones, la posibilidad de seguir a otros buceadores con perfiles públicos, y un centro de notificaciones que informa al usuario de toda actividad relacionada con su contenido.

10.1 CARACTERÍSTICAS DESARROLLADAS ESTA ITERACIÓN

1. Interacciones con publicaciones: likes, guardados y comentarios con soporte para menciones a otros usuarios. Ver Tabla §10.1.
2. Seguimiento de usuarios y perfiles públicos: seguir a otros buceadores y consultar su perfil con contadores de seguidores y seguidos. Ver Tabla §10.2.
3. Centro de notificaciones: campana con contador de no leídas que agrupa likes, comentarios, menciones y nuevos seguidores. Ver Tabla §10.3.

Análisis de valor aportado: Interacciones con Publicaciones	
Propuesta	Añadir botones de like y guardado a cada publicación del mapa, y una sección de comentarios con soporte para mencionar a otros usuarios por nombre.
Valor	Da retroalimentación directa al autor y ayuda a otros usuarios a identificar el contenido más interesante. El guardado permite construir una lista personal de inmersiones que el usuario quiere visitar. Los comentarios abren la posibilidad de preguntar detalles, compartir experiencias propias en ese mismo punto o mencionar a alguien que puede estar interesado.
Coste	Tres tablas nuevas en base de datos (likes, guardados, comentarios), lógica de toggle para que el mismo botón active y desactive la acción sin duplicar registros, y análisis del texto de los comentarios para resolver las menciones y notificar a los usuarios citados.
Opciones	Solo likes sin comentarios: más simple de implementar, pero elimina la conversación entre buceadores que enriquece el contenido. Reacciones con emojis: más expresivas, pero añaden complejidad de diseño e implementación sin un beneficio claro para la comunidad de buceo.
Riesgos	Sin moderación ni filtrado de contenido, cualquier texto puede publicarse como comentario. No hay control sobre el volumen de comentarios por publicación.
Deuda técnica	Sin paginación de comentarios. Sin posibilidad de editar un comentario ya publicado. Sin sistema de denuncias de contenido inapropiado.

Cuadro 10.1: Análisis de valor aportado: Interacciones con Publicaciones

Análisis de valor aportado: Seguimiento de Usuarios y Perfiles Públicos	
Propuesta	Permitir seguir a otros usuarios y consultar su perfil público con nombre, foto y contadores de seguidores y seguidos.
Valor	Construye la dimensión social de Scubex. Un buceador puede seguir a quien publica habitualmente en sus zonas favoritas y, con el tiempo, formar parte de una comunidad sin necesidad de conocerse fuera de la aplicación. El perfil público da identidad a cada autor: el nombre y la foto dan contexto a las publicaciones del mapa.
Coste	Tabla de follows con comprobación de que un usuario no puede seguirse a sí mismo, endpoint de perfil público accesible, componentes de listas de seguidores y seguidos, y actualización optimista en la interfaz para que el botón responda de forma inmediata sin esperar al servidor.
Opciones	Amistad bidireccional: ambas partes deben aceptarse mutuamente, lo que es más privado pero pierde la asimetría característica de las redes de interés. Sin sistema de follows: el contenido sería completamente anónimo y no habría forma de seguir a un autor concreto ni de construir comunidad.
Riesgos	Los perfiles son completamente públicos: nombre, foto y publicaciones son visibles para cualquier visitante sin necesidad de cuenta. No hay opción de perfil privado.
Deuda técnica	Sin perfiles privados ni opción de aprobar seguidores. Sin bloqueos entre usuarios.

Cuadro 10.2: Análisis de valor aportado: Seguimiento de Usuarios y Perfiles Públicos

Análisis de valor aportado: Centro de Notificaciones	
Propuesta	Campana en la barra de navegación con contador de no leídas que abre un panel con las notificaciones de likes, comentarios, menciones y nuevos seguidores; cada una puede eliminarse deslizándola.
Valor	El usuario sabe si alguien interactuó con su contenido sin tener que revisar manualmente cada publicación. El contador visible en la campana mantiene el engagement con la aplicación sin requerir recarga de página.
Coste	Tabla de notificaciones con cuatro tipos (like, comentario, mención, follow) y consulta periódica del contador desde el frontend.
Opciones	Sin notificaciones: el usuario solo descubre las interacciones si revisa manualmente sus propias publicaciones, lo que reduce significativamente el engagement.
Riesgos	El intervalo de polling introduce un pequeño retraso entre el evento y la notificación visible. Sin agrupación, una publicación popular puede generar un gran número de entradas individuales en el panel.
Deuda técnica	Sin notificaciones push al navegador cuando la pestaña está en segundo plano. Notificaciones del mismo tipo no se agrupan: diez likes de distintos usuarios generan diez entradas separadas. No existe opción de silenciar notificaciones por tipo ni por publicación concreta.

Cuadro 10.3: Análisis de valor aportado: Centro de Notificaciones

10.2 DISEÑO

El diagrama UML de la Fig. §10.1 muestra los componentes de la capa social: controladores de interacciones y notificaciones, servicios, repositorios y entidades, así como las dependencias entre `InteractionService` y `NotificationService`.

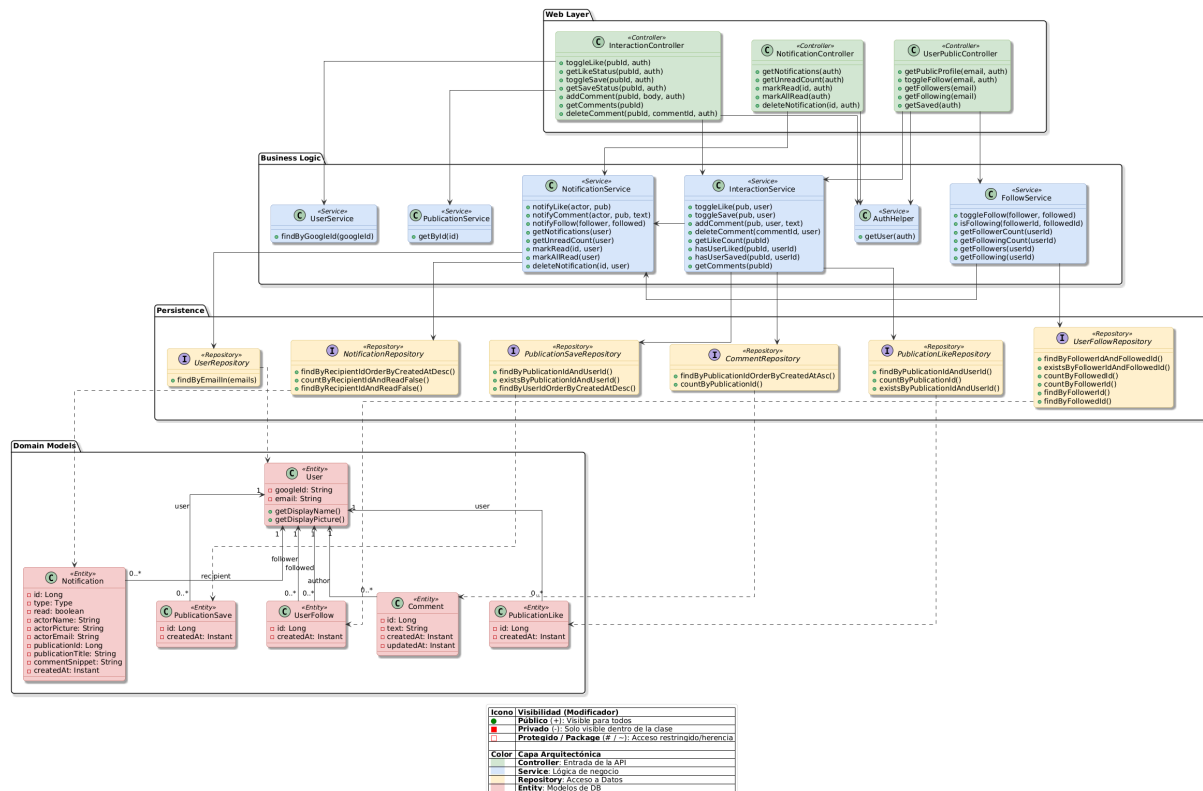


Figura 10.1: Diagrama UML de diseño para la iteración 4

Nota: `NotificationRepository` expone además los métodos `existsByRecipientIdAndTypeAndActorEmail()`, `existsByRecipientIdAndTypeAndActorEmailAndPublicationId()` y `deleteAllByRecipientId()`, omitidos del diagrama para no comprometer la legibilidad.

10.3 IMPLEMENTACIÓN

Se presentan primero los memorandos técnicos con las decisiones de diseño; a continuación, los pseudocódigos que las materializan.

Memorando técnico 012: Implementación de Likes, Guardados y Comentarios

Asunto	Las publicaciones admiten tres tipos de interacción: like, guardado y comentario, cada uno con su propio botón. Likes y guardados pueden deshacerse; los comentarios se añaden siempre como entrada nueva.
Resumen	Para likes y guardados, el servidor mira si ya existe el registro antes de actuar: si ya está, lo borra; si no, lo crea. Así nunca se acumulan registros incorrectos. Los comentarios siempre generan una entrada nueva y solo el propio autor puede borrarlos.
Factores causantes	Sin la comprobación previa, dar like varias veces crearía varios registros en base de datos y dispararía una notificación al autor por cada uno, aunque el usuario simplemente hubiera pulsado el botón dos veces.
Solución	Cada publicación expone tres botones independientes: uno para dar o quitar like, uno para guardar o dejar de guardar, y una sección de comentarios con su propio campo de texto. Para likes y guardados, el servidor busca si ya existe un registro para ese usuario y esa publicación: si lo encuentra, lo borra; si no, lo crea. El like además notifica al autor, excepto cuando el autor se da like a sí mismo. Los comentarios siempre generan una entrada nueva; el borrado comprueba que quien borra sea el autor del comentario: si no lo es, el servidor no toca nada.
Motivación	El patrón de botón con toggle es el estándar en redes sociales: Instagram, Twitter y similares usan exactamente este modelo para likes y guardados. Como ventaja adicional, el frontend no necesita conocer el estado actual antes de llamar al servidor: simplemente envía la petición y refleja el resultado que recibe.
Cuestiones abiertas	Los comentarios no tienen paginación: si una publicación acumula muchos, todos se devuelven de golpe. No hay opción de editar un comentario ya publicado.
Alternativas	Endpoints separados por acción: un endpoint para dar like, otro para quitarlo, otro para guardar, otro para borrar guardado, etc. Más explícito, pero multiplica el número de endpoints y obliga al frontend a conocer el estado actual antes de cada llamada para saber cuál invocar.

Identificador	Descripción de la acción de alto nivel			
006	InteractionService			
Métodos de alto nivel				
boolean toggleLike(publication:Publication, user:User)				
Pasos				
1. Se busca en base de datos si el usuario ya ha dado like a esta publicación con likeRepository.findByPublicationIdAndUserId().				
2. Si el like ya existe, se elimina con likeRepository.delete() y se devuelve false (el usuario ha quitado el like), fin.				
3. Si no existe, se crea el registro de like y se persiste con likeRepository.save().				
4. Se notifica al autor de la publicación con notificationService.notifyLike(). Si el usuario está dando like a su propia publicación, esta llamada no genera ninguna notificación.				
5. Se devuelve true (el usuario ha dado like).				
Métodos de bajo nivel necesarios				
Paso	Clase	Método	Mem. Técn.	IU
1	LikeRepo.	Optional<PublicationLike> findByPublicationIdAndUserId(pubId, userId)	012	NO
2	LikeRepo.	void delete(like)	012	NO
3	LikeRepo.	PublicationLike save(like)	012	NO
4	NotifSvc.	void notifyLike(actor, pub)	012, 013	NO

Nota: Los métodos restantes de InteractionService son suficientemente simples como para no requerir pseudocódigo propio. toggleSave() sigue el mismo patrón que toggleLike() sin el paso de notificación. addComment() persiste el comentario y delega en notifyComment() (ver asigResp. 007). deleteComment() comprueba autoría y borra, devolviendo false si el usuario no es el autor.

Memorando técnico 013: Notificaciones con Menciones en Comentarios	
Asunto	Cada acción social genera una notificación para el destinatario correspondiente. Los comentarios añaden la posibilidad de mencionar a otros usuarios, que también deben ser notificados sin que nadie reciba dos notificaciones por el mismo evento.
Resumen	El servidor identifica al destinatario de cada acción y persiste la notificación. En comentarios, extrae los emails mencionados, carga los usuarios y lleva la cuenta de quién ya fue notificado para no repetir.
Factores causantes	Los nombres de usuario no son únicos, así que las menciones no pueden resolverse por nombre. Además, si el propietario de la publicación también aparece mencionado en el comentario, sin control recibiría dos notificaciones por el mismo evento.
Solución	El frontend codifica las menciones como <code>@[Nombre](email)</code> , usando el correo como identificador. Al publicar un comentario, el servidor extrae los emails mencionados en el texto, carga esos usuarios en una sola consulta y envía a cada uno una notificación, saltándose al propietario de la publicación. Para likes y follows, si el usuario da like, lo quita y lo vuelve a dar, solo la primera vez debe llegar una notificación al autor; las siguientes se descartan comprobando en base de datos si ya existe una notificación igual.
Motivación	Una sola consulta para todos los mencionados evita una consulta por cada mención. El conjunto de ya-notificados garantiza que nadie recibe dos notificaciones por el mismo comentario sin complicar el modelo de datos.
Cuestiones abiertas	Cualquier usuario puede escribir manualmente el patrón <code>@[Nombre](email)</code> en el campo de comentarios con el correo de cualquier persona registrada, y el servidor le enviará una notificación de mención sin que forme parte de la conversación.
Alternativas	Sin menciones: simplifica la implementación pero elimina una forma directa de dirigirse a otro usuario en un comentario. Se descartó porque los usuarios piloto solicitaron explícitamente esta funcionalidad en varias ocasiones durante las pruebas.

Memorando técnico 014: Centro de Notificaciones con Polling desde el Frontend	
Asunto	El usuario debe ver las notificaciones nuevas sin tener que recargar la página.
Resumen	La campana de notificaciones consulta periódicamente al servidor cuántas notificaciones sin leer hay y actualiza el contador (polling). Al abrir el panel, descarga la lista completa y marca todas como leídas.
Factores causantes	Las notificaciones las generan acciones de otros usuarios, por lo que el cliente no sabe cuándo llega una nueva. Sin consultas periódicas, el contador solo cambiaría al recargar la página.
Solución	La campana pregunta al servidor el número de no leídas cada cierto intervalo y actualiza el badge. Al hacer clic, descarga las notificaciones y las marca como leídas. Cada notificación puede eliminarse individualmente deslizándola. Si la notificación está relacionada con una publicación, al pulsar sobre ella el mapa navega directamente a esa publicación.
Motivación	El polling no requiere conexiones permanentes en el servidor y es suficiente para el volumen de usuarios esperado en Scubex. La latencia introducida por el intervalo de consulta es poco relevante en la práctica.
Cuestiones abiertas	El intervalo de polling fija el retraso máximo con el que el usuario ve una notificación nueva. No hay alertas en el sistema operativo cuando la pestaña está en segundo plano.
Alternativas	WebSockets: tiempo real completo, pero una mayor complejidad de infraestructura, innecesaria para la repercusión que tendría en la aplicación.

Cuadro 10.6: Memorando técnico 014: Centro de Notificaciones

Identificador	Descripción de la acción de alto nivel			
007	NotificationService			
Métodos de alto nivel				
void notifyComment(actor:User, pub:Publication, text:String)				
Pasos				
1. Se prepara el fragmento del texto para la notificación, recortado a 150 caracteres si el comentario es más largo.				
2. Se inicializa un conjunto de ya-notificados con el ID del comentarista, para que nunca reciba él mismo una notificación de tipo COMMENT ni de tipo MENTION.				
3. Si el comentarista no es el propietario, se persiste una notificación de tipo COMMENT para el propietario con notificationRepository.save() y se añade su ID al conjunto, evitando que reciba también una MENTION si aparece mencionado en el texto.				
4. Se extraen todos los emails mencionados del texto con MENTION_PATTERN.				
5. Los usuarios mencionados se cargan en una sola consulta con userRepository.findByEmailIn(), evitando una consulta separada por cada mención.				
6. Por cada mencionado cuyo ID no figure en el conjunto de ya-notificados, se persiste una notificación de tipo MENTION con notificationRepository.save().				
Métodos de bajo nivel necesarios				
Paso	Clase	Método	Mem. Técn.	IU
3	NotifRepo.	Notification save(notification)	012, 013	NO
5	UserRepo.	List<User> findByEmailIn(emails)	013	NO
6	NotifRepo.	Notification save(notification)	013	NO

Nota: Por la misma razón de la anterior nota, `notifyLike()` y `notifyFollow()` no tienen pseudocódigo propio. `notifyLike()` descarta la notificación si el actor es el autor de la publicación o ya existe una igual, y en caso contrario persiste una de tipo LIKE. `notifyFollow()` sigue el mismo patrón sin el filtro de autoría.

10.4 PRUEBAS

Las pruebas unitarias de esta iteración se reparten en dos suites: `InteractionServiceTest` y `NotificationServiceTest`, siguiendo el mismo patrón basado en Mockito que el resto del proyecto.

`InteractionServiceTest` cubre los siguientes casos:

- **Primer like** (`toggleLike_noExisting_likesAndNotifies`): si no existe like pre-

vio, se persiste el registro y se llama a `notificationService.notifyLike()`.

- **Quitar like** (`toggleLike_existing_unlikesAndDoesNotNotify`): si el like ya existe, se elimina y `notifyLike()` no es invocado.
- **Contador de likes** (`getLikeCount_returnsRepositoryValue`): el servicio delega directamente en el repositorio y devuelve el valor sin transformación.
- **Primer guardado y quitar guardado** (`toggleSave_noExisting_saves` y `toggleSave_existing_unsaves`): el mismo patrón que los likes, verificando que el registro se crea o se elimina según corresponda.
- **Añadir comentario** (`addComment_savesCommentAndNotifies`): se persiste el comentario y se llama a `notificationService.notifyComment()`.
- **Borrar comentario por propietario ajeno** (`deleteComment_notOwner_returnsFalse`): si el usuario que intenta borrar no es el autor del comentario, devuelve `false` sin llamar a `delete()`.
- **Borrar comentario inexistente** (`deleteComment_notFound_returnsFalse`): si el comentario no existe en base de datos, devuelve `false` sin intentar borrarlo.

`NotificationServiceTest` cubre los siguientes casos:

- **Notificación de follow** (`notifyFollow_differentUsers_savesNotification`): seguir a otro usuario persiste una notificación de tipo `FOLLOW` con el email del seguidor.
- **Follow ya existente** (`notifyFollow_alreadyNotified_doesNotDuplicate`): si ya existe una notificación de tipo `FOLLOW` del mismo actor para el mismo destinatario, `existsByRecipientIdAndTypeAndActorEmail()` devuelve `true` y no se persiste ninguna notificación adicional.
- **Like ya notificado** (`notifyLike_alreadyNotified_doesNotDuplicate`): si ya existe una notificación de tipo `LIKE` del mismo actor para la misma publicación, `existsByRecipientIdAndTypeAndActorEmailAndPublicationId()` devuelve `true` y no se persiste ninguna notificación adicional.
- **Like propio** (`notifyLike_selfLike_doesNotSave`): dar like a la propia publicación no genera notificación.

- **Comentario con mención** (`notifyComment_withMention_savesMentionNotification`): un comentario que menciona a un usuario produce dos saves: uno de tipo COMMENT para el propietario y otro de tipo MENTION para el mencionado.
- **Mención al propietario sin duplicar** (`notifyComment_mentionAlreadyNotified_doesNotDuplicate`): si el propietario de la publicación también es el mencionado, solo se genera un save, no dos.
- **Texto largo truncado** (`notifyComment_longText_snippetTruncatedAt150`): un comentario de más de 150 caracteres produce un fragmento de exactamente 151 (150 + "...").
- **Marcar notificación propia como leída** (`markRead_ownNotification_setsReadAndSaves`): la notificación queda marcada como leída y se persiste el cambio.
- **Marcar notificación ajena** (`markRead_otherUserNotification_doesNotSave`): solo el destinatario puede marcar su notificación; si el usuario no es el destinatario, no se persiste ningún cambio.
- **Marcar todas como leídas** (`markAllRead_setsAllReadAndSaves`): todas las notificaciones no leídas del usuario quedan marcadas y se persisten en lote.
- **Eliminar notificación propia** (`deleteNotification_ownNotification_deletes`): el destinatario puede eliminar su notificación correctamente.

10.5 DESPLIEGUE

Esta iteración añadió cinco nuevas tablas a la base de datos: `publication_likes`, `publication_saves`, `comments`, `user_follows` y `notifications`, todas creadas automáticamente por Hibernate al arrancar el servicio en Railway con `ddl-auto=update`. No fue necesario añadir nuevas variables de entorno ni credenciales externas, ya que todas las funcionalidades de esta iteración utilizan exclusivamente la base de datos PostgreSQL existente.

PARTE IV

CIERRE DEL PROYECTO

MANUAL DE USUARIO

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

11.1 SECCIÓN LIBRE

Estructurar en función del proyecto.

CONCLUSIONES

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

12.1 INFORME POST-MORTEM

Qué es un informe post-mortem

12.1.1 Lo que ha ido bien

- Argumento a favor 1.
- Argumento a favor 2.
- Argumento a favor 3.

12.1.2 Lo que ha ido mal

- Argumento en contra 1.
- Argumento en contra 2.
- Argumento en contra 3.

12.1.3 Discusión

En función de lo anterior, qué cambiaría si empezara hoy el proyecto de nuevo.

12.2 TRABAJOS FUTUROS

Enumera los puntos abiertos y que no se han resuelto. Indica si darían lugar a otro proyecto y de qué forma se podría acotar.

PARTE V

APPENDICES

SOFTWARE PRODUCT LINES

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

***T**his is an example of an abstract. Multiple lines are supported. Several paragraphs. It jumps to the next page. Blau blau blau. I am introducing more text to reach the third line*

A.1 SOFTWARE PRODUCT LINES

- Objective of a Product Line (PL) (mass production and customisation) [1]
- The focus in software derives in Software Product Lines (SPLs).
- Variability management: variability models
- When and how are used VMs: FMs are described in FODA report as a key element in SPL since they represent the variability and commonality of the different products in a SPL.

A.2 FEATURE MODELS

To Abductive Section in 2.1

As the number of products to be built by a SPL may be large and the constraints among features may be complex, representing such an information in a manageable and compact manner is a must. Feature Models (FMs) represent the set of products a SPL may build in terms of product features. Some features are optional while others are mandatory. To indicate the relationships among features, they are hierarchically linked, forming a tree whose root is a feature representing the whole functionality of a product. The root feature is refined in child features, which increase the level of detail and reduce the scope of features. Recursively following this refinement process, a tree-like structure is obtained where three basic kinds of hierarchical relationships are used:

- **Mandatory:** a mandatory relationship affects a parent and child feature. It forces the child feature to appear in a product whenever its parent feature does.
- **Optional:** a child feature connected to a parent feature by means of an optional relationship may be optionally selected whenever its parent feature is.
- **Set-relationships:** three or more features are part of a set-relationship: a parent feature and a set of two or more child features. A set-relationship contains a cardinality that constraints the number of child features to be selected in a product whenever its parent feature is selected. If the cardinality is $[1,1]$ it is commonly remarked as an *alternative relationship* where only one child feature may be selected at the same time. If the cardinality is $[1..N]$ (where N is the number of

child features), it is also known as an *or-relationship* as any combination of child features is allowed while at least one is selected.

Although FMs can represent most of the most frequent constraints, the hierarchical nature of these models might hinder the representation of some constraints. Under this circumstance, *cross-tree constraints* can be added. The most common kinds of cross-tree constraints are:

- Dependency: a feature depends on another feature if the second one must be part of a product whenever first one is selected.
- Exclusion: two features exclude themselves if both of them cannot be part of a product at the same time.

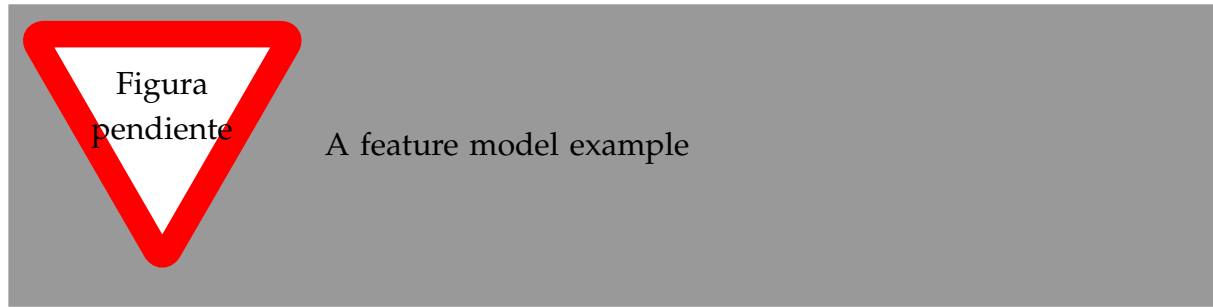


Figura A.1: An example of a Home Integration System

The example in Figure §A.1 describes a *Home Integration System* (HIS) SPL in terms of its features and the relationships among them. Leaning on this example we define some useful terms:

Partial configuration : a partial configuration is a composed by three sets of selected (S), removed(R) and undecided(U) features. A feature can only be in one of these sets and every feature in the FM (fm) must be in one of them, i.e. $S \cup R \cup U = fm$ and $S \cap R \cap U = \emptyset$. A partial configuration represents an intermediate state during the process of a customer selecting the feature for a custom product. For example, $S_P = \{\dots\}$, $R_P = \{\dots\}$ and $U_P = \{\dots\}$ define a partial configuration for the sample FM where some features are still to be decided if they are to be selected or removed in a configuration.

(Full) configuration : a full configuration or simply a configuration is a partial configuration such that the set of undecided features is empty. For example, $S_F = \{\dots\}$ and $R_F = \{\dots\}$ describe a full configuration for the example FM.

Product : a product is a representation for a full configuration such that only the selected features are remarked. For instance, $P = \{\}$ is a product for the above full configuration. A product such as A,B is a valid since all the constraints within the FM are satisfied. However, A,B and C is not a valid product since D is required.

Validation A partial configuration is *valid* if all the relationships and constraints are satisfied given the sets of selected, removed and undecided features. So the definition applies for valid full configurations and valid products. As a conclusion we can affirm that a FM represents all the valid products in a SPL.

Objetivo: Briefly expose attributes as an important asset in feature models.

It is frequent that features are not enough to represent information that is relevant to represent a SPL variability. In this case, FMs are extended with feature attributes such as cost, versions, RAM consumption, etc. in the so-called Extended Feature Models (EFMs) [1]. Besides relationships, an EFM contains constraints that affect attributes which reduce even more the set of products a FM describes. Above definitions remain when attributes are introduced into FMs.

A.3 AUTOMATED ANALYSIS OF FEATURE MODELS

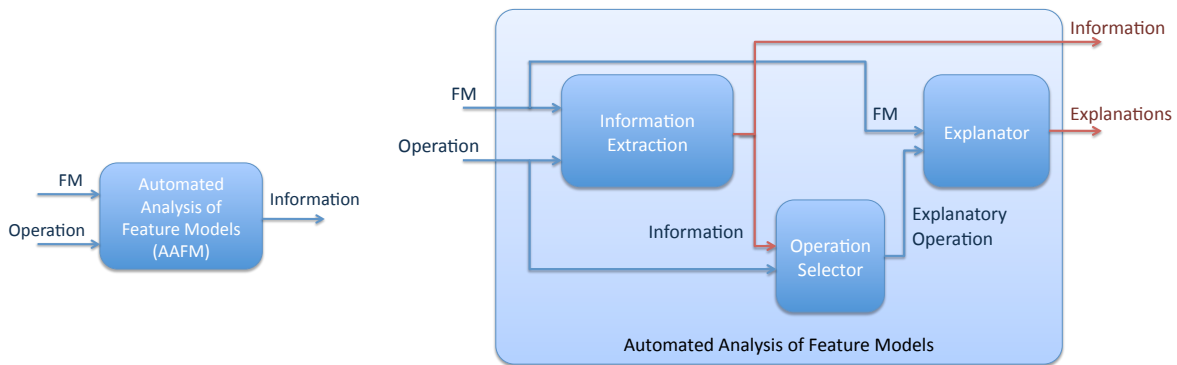
A.3.1 Scope

To Abductive Intro

FMs are used all along the SPL development as key models and many of the development decisions are taken relying on the information contained within them. Most of the times, relationships are complex and hinder the manual extraction of information. Manually obtaining information such as ‘which is the product that costs the less?’, ‘does the feature model contain errors?’ or ‘why there exist no product containing certain features?’ can be an unfeasible task. The complexity and compactness of FMs justify the need of an automated support of these operations. So the *Automated Analysis of Feature Models* (AAFM) arises as a topic of interest to deal with this problem in the SPL community.

The AAFM can be seen as a black-box process that receives a FM and an operation as inputs and obtains information (its kind depends on the analysis operation) as an

output (Fig. A.2(a)). There are many operations that extract information from a FM such as 'counting products' operation whose result is a natural number indicating the number of customised products that can be built; or 'list of products' operation that obtains each of those products. This vision of AAFM as a black-box is valid for a subset of analysis operations that we call *information extraction operations* (IEO) that can be seen as processes to extract information from FMs. In other words, an IEO makes explicit an implicit information within a FM.



(a) The AAFM seen as a black-box process

(b) Extending the AAFM process with explanations

Figura A.2: A different view on AAFM distinguishing between information extraction and explanatory operations

Use me to explain in a larger text than 'side-text' anything that is important to a reader not familiar with the dissertation context for example.

However, there is a subset of analysis operations known as *explanatory operations* (EO) whose objective is explaining the result obtained from a IEO. Sometimes, the result is not the expected one and the analyser needs to know which are the relationships that have caused it. For example, let us suppose that the IEO 'which are the products described in a FM that cost less than \$1000?' obtains no products as a result. If we were expecting to obtain at least one product, it is important to determine the relationships in the FM that are responsible of that behaviour, so an EO 'why there is no product costing less than \$1000?' will shed light on the relationships that avoid obtaining any product. Obtaining no result is not the only case that claims for explanations.

If we obtained only one product as a result and we were expecting to obtain at least 10 products, although an answer is obtained the result is unexpected and the discrepancy reasons have to be found. Moreover, explanatory operations are also use-

ful even when an expected result is obtained, to reinforce the certainty that the result is correct. So it can be concluded that EOs complement the information an FM analyser obtains from IEOs.

The complexity of feature modelling relies on correctly setting the relationships that describe the set of products to be built in a SPL. Relationships are the only elements responsible of the results obtained in FM analysis. So an *explanation* is a set of relationships that may have caused that result. While IEO provides for an unique response that is known for certain, an EO provides for a set of probable explanations to a result obtained from a IEO, being only one of them a valid explanation. It would be the analyser the one in charge of discriminating the correct explanation, maybe performing new analysis operations.

**THIS IS A SIDE TEXT. USE TO
REMARK IMPORTANT
INFORMATION**

Therefore, two kinds of operations are distinguished in AAFM: information extraction and explanatory operations. Explanatory operations have no sense without a paired information extraction operation and its result. To ensure that explanatory operations are always paired to an information extraction operation, we define a new black-box process of AAFM that incorporates explanations as an additional output (see Figure A.2(b))

1. Information extraction: the original process, which remains the same.
2. Operation selector: depending on the information extraction operation the analyser asks for and the information obtained as a result, this process provides the explanatory operation to be performed. In other words, it pairs an explanatory operation to an information extraction operation.
3. Explanatory analysis: provides a set of explanations from the FM and the explanatory operation.

The overall process can be encapsulated into a holistic black-box process which receives the FM and the information extraction operation as inputs and provides a result and explanations as outputs. It can be seen as we just add explanations as an output to the analysis process.

To realise this view on the AAFM, we need to give details on the insides of these black-boxes. Since the information extraction process is already rigourously defined in

Benavides' PhD dissertation, the purpose of this paper is defining the remaining two sub-processes. We formalise the explanatory analysis process by means of default logic and provide the criteria to implement the operation selector process.

Most Common Techniques to perform AAFM Operations.

A.4 DYNAMIC SOFTWARE PRODUCT LINES (DSPL)

What is a Dynamic Software Product Line (DSPL). Different points of view. What is important is the automation of reconfiguration properties relying on SPL techniques.

We focus in the application of explanations in DSPLs as an application of our results. Specifically we have worked in MAS and smart homes providing a solution for automating product reconfiguration.

A.5 HYPOTHESIS AND OBJECTIVES

Objetivo: Justifying that explanations are a particular set of operations in AAFM that are not solvable by means of the techniques that are used up-to-date

Objetivo: Set an impacting phrase that summarises the hypothesis

Hypothesis

*Explanations cannot be solved by AI techniques used to solve AAFM.
There should exist other AI techniques to solve explanations.*

Objective of the dissertation

Defining a framework to provide solutions for explanatory analysis in FMs.

This dissertation summarises our contribution to solve some of the objectives we set in our PhD project.

- Defining a catalog of analysis operations where explanations are applied.
- Rigorously defining these operations in terms of logics.
- Proposing solutions to these operations.
- Validating our results by means of tools and projects where they are applied.

Next chapter focuses on refining how we have contributed to deal with the above objectives.

A piece of code...

```
public Map<Cardinality, CardinalValue> detectWrongCardinals() {
    // any other implementation of Map can be used instead.
    Map<Cardinality, CardinalValue> result =
        new TreeMap<Cardinality, CardinalValue>();
    for( r : relationships) {
        if (r instanceof Set) {
            Set set = (Set)r;
            Cardinality card = set.getCardinality();
            Domain dom = card.getDomain();
            for (value: dom.getValues())
                if (isWrongCardinal(card, value))
                    result.put(card, value);
        }
    }
    return result;
}
```

A coolTable. Use inside a table.

Use `\TableSubtitle{n,title}` to add a subtitle as the header. *n* is the number of columns and *title* is the text to place. [1]

A Catalog of FM Explanatory Operations (2009 version)		
Information Extraction Operation	FM Explanatory Operations	
	<i>Why? operation</i>	<i>Why not? operation</i>
Valid FM	-	invalid FM
Valid Configuration	valid partial conf.	invalid partial conf.
Valid Product	valid product	invalid product
Products Listing	vaild Product/Config	invalid FM/Product/Config
Products Counting	vaild Product/Config	invalid FM/Product/Config
Optimisation	vaild Product/Config	invalid FM/Product/Config
Core feature	core feature	core feature
Variant feature	variant feature	variant feature
Dead feature detection	-	dead feature
False-optional feature detection	-	false-optional feature
Wrong-cardinality detection	-	wrong cardinal
Information Extraction Operation	Configuration Explanatory Operations	
	<i>Why? operation</i>	<i>Why not? operation</i>
Valid Configuration	valid partial conf.	invalid partial conf.

Cuadro A.1: Most frequently used explanatory operations and their corresponding information extraction operations

BIBLIOGRAFÍA

- [1] D. Benavides, A. Ruiz-Cortés, and P. Trinidad. Automated reasoning on feature models. *LNCS, Advanced Information Systems Engineering: 17th International Conference, CAiSE 2005*, 3520:491–503, 2005. ISSN 0302-9743. (pages 98, 100 y 104).